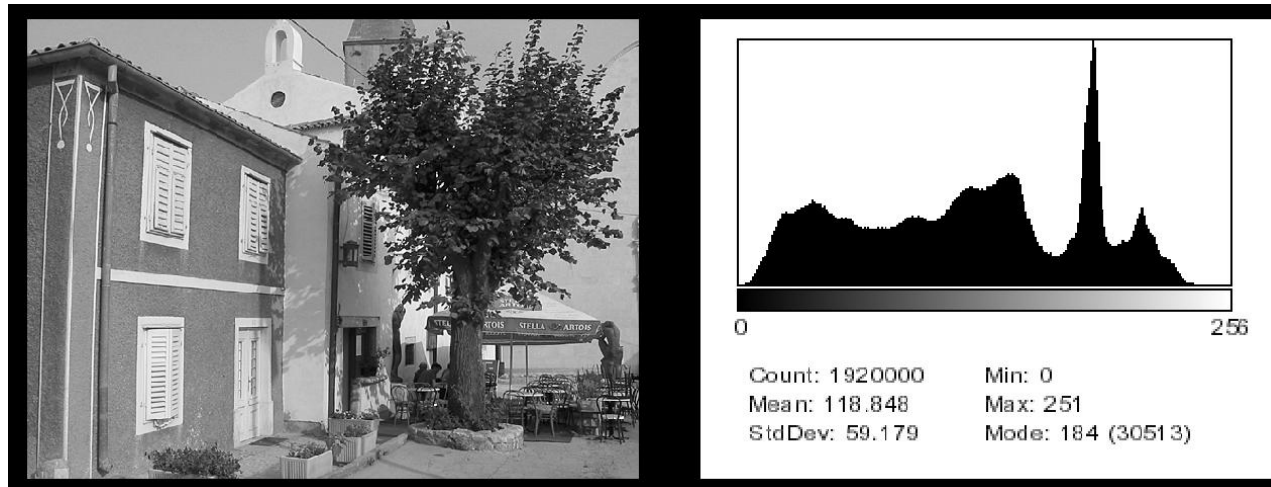


- + • **HISTOGRAM,**
- **IMAGE ENHANCEMENT**

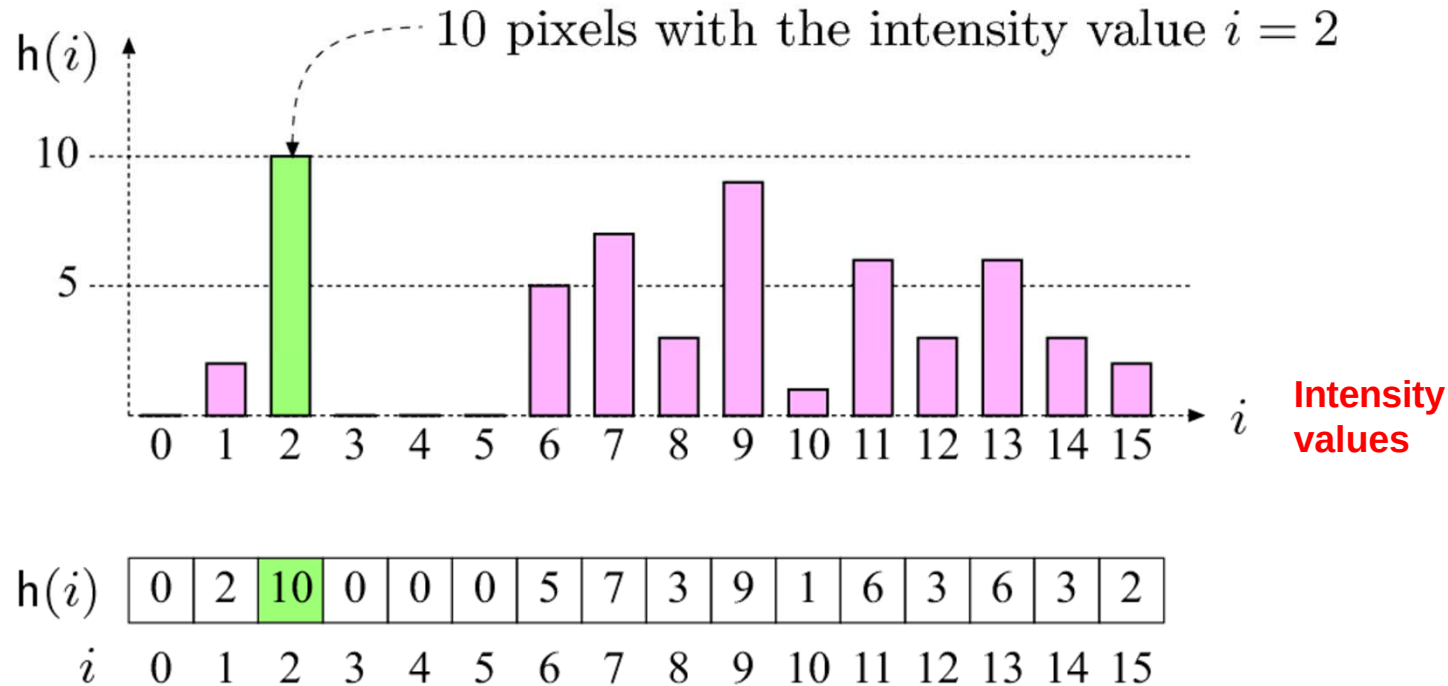
Nada Fitrieyatul Hikmah

Histograms

- Histograms plots how many times (frequency) each intensity value in image occurs
- Example:
 - Image (left) has 256 distinct gray levels (8 bits)
 - Histogram (right) shows frequency (how many times) each gray level occurs



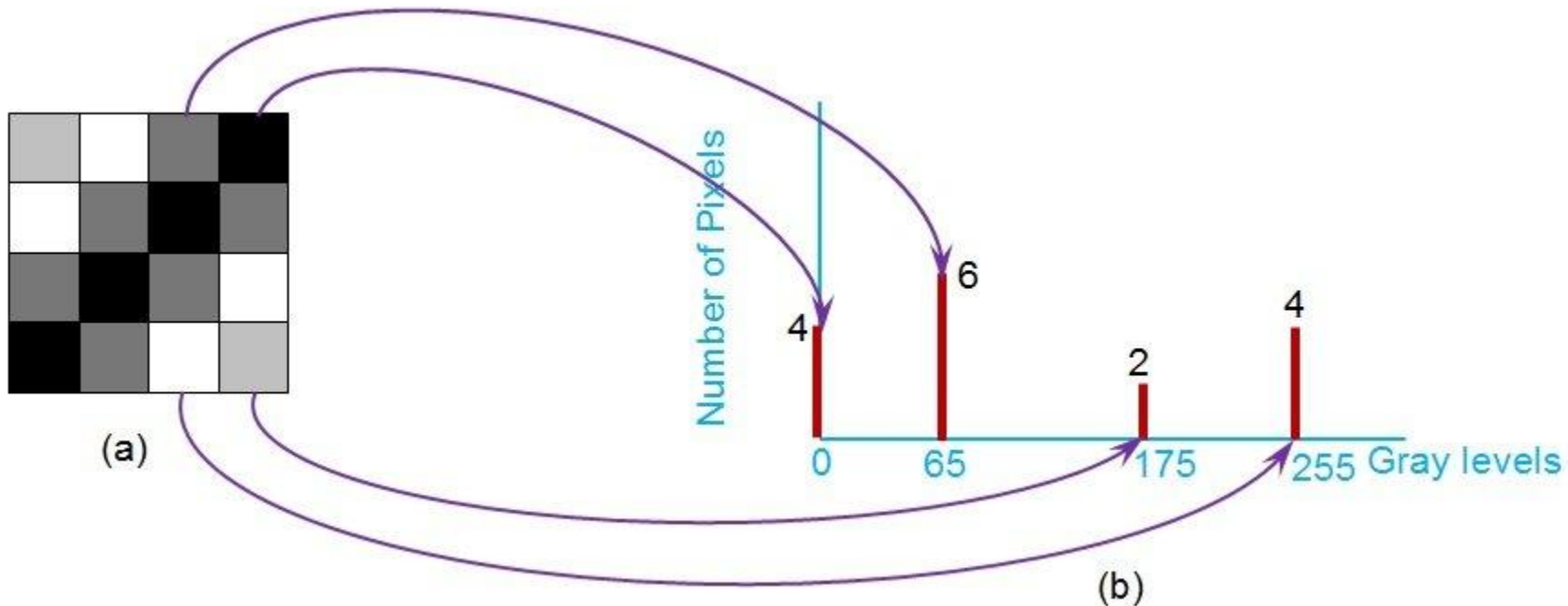
Histograms



- E.g. $K = 16$, 10 pixels have intensity value = 2
- Histograms: only statistical information
- No indication of **location** of pixels

Histogram

- **Histogram:** count number of pixels at each intensity value.
- Implemented in `scipy.ndimage` :
 - higher-dimensional arrays
 - masked data



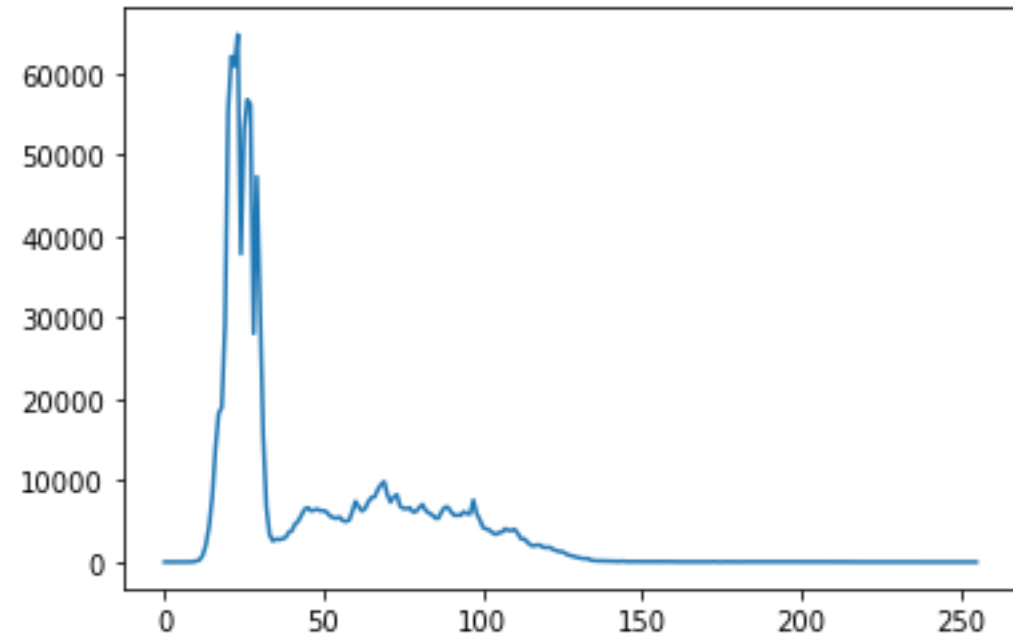
Histogram in Python (using scipy)

```
1 import imageio
2 import matplotlib.pyplot as plt
3
4 # Load the hand radiograph
5 im = imageio.imread('D:\Teaching\Pengolahan Ci
6 print('Data type:', im.dtype)
7 print('Min. value:', im.min())
8 print('Max value:', im.max())
9
10 # Plot the grayscale image
11 plt.imshow(im, cmap='gray', vmin=0, vmax=255)
12 plt.axis('off')
13 plt.colorbar();
```

Data type: uint8
Min. value: 8
Max value: 236

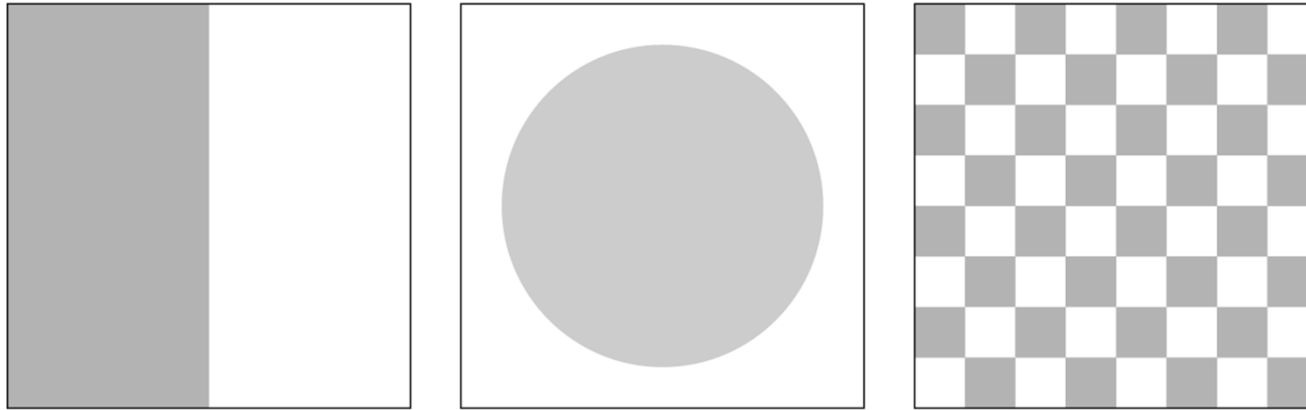


```
1 import scipy.ndimage as ndi
2 hist = ndi.histogram(im, min=0, max=255, bins=256)
3 plt.plot(hist)
4 plt.show()
```



Histograms

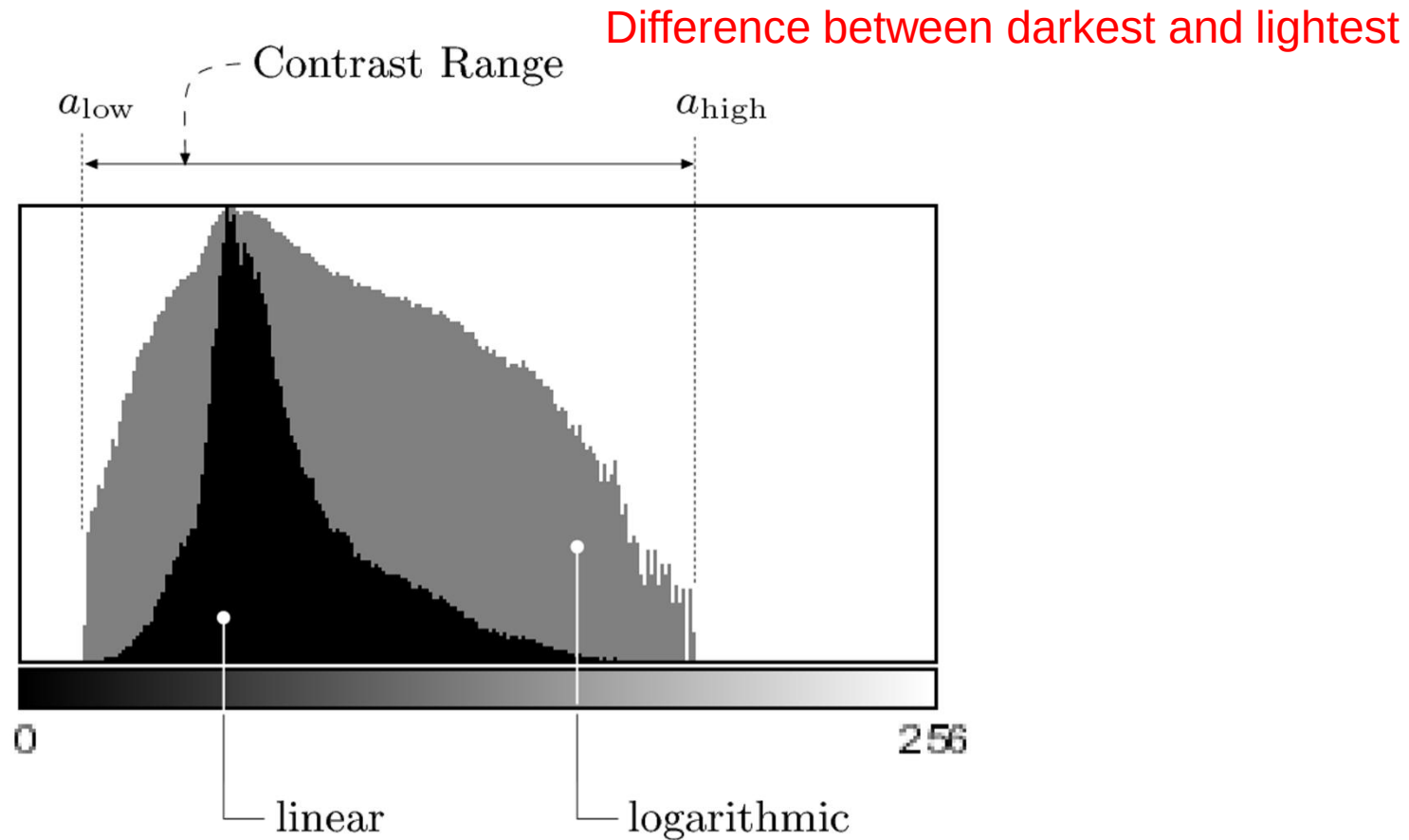
- Different images can have **same histogram**
- 3 images below have same histogram



- Half of pixels are gray, half are white
 - Same histogram = same statistics
 - Distribution of intensities could be different
- Can we reconstruct image from histogram? No!

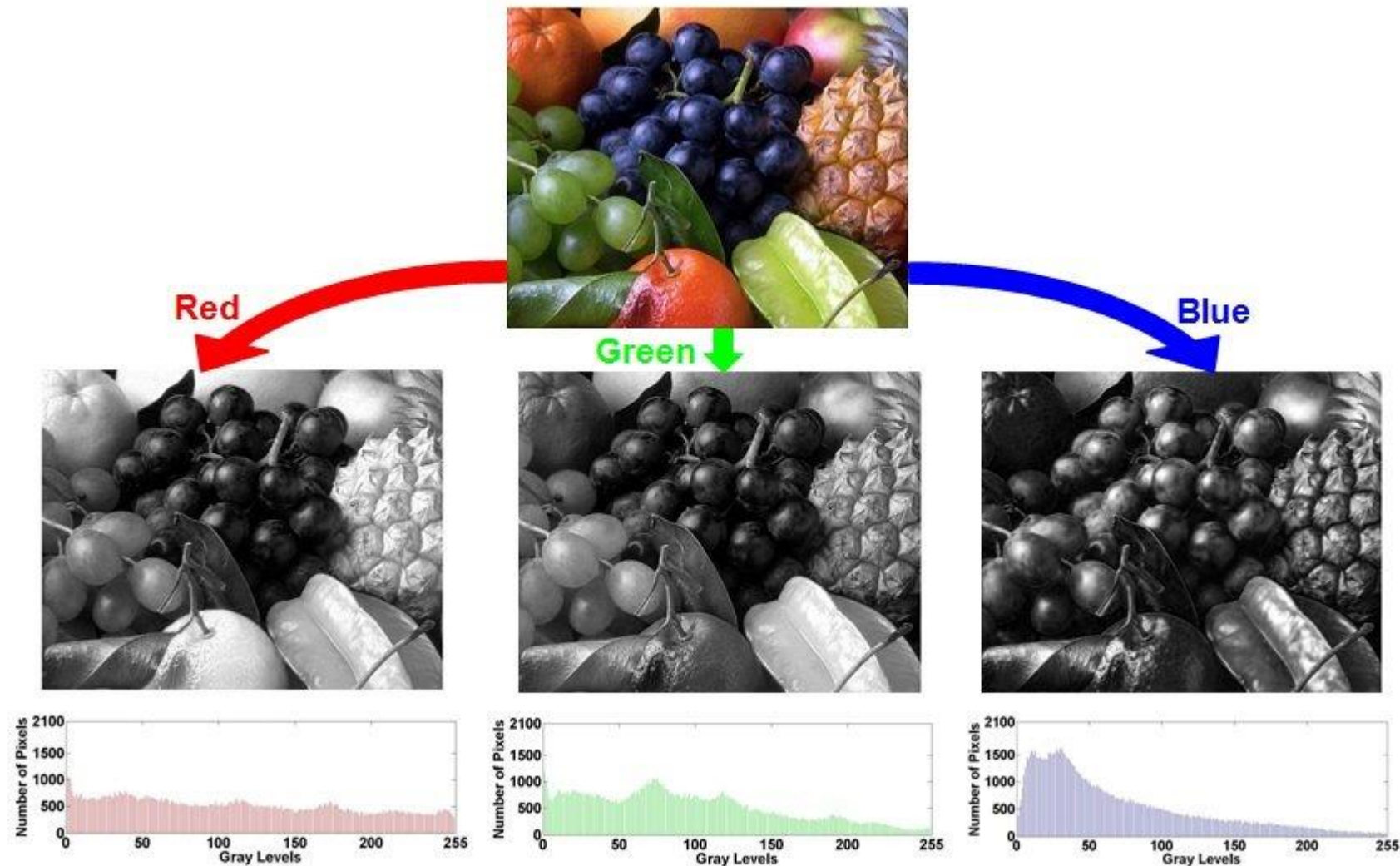
Interpreting Histograms

- Log scale makes low values more visible



Histogram of RGB Image

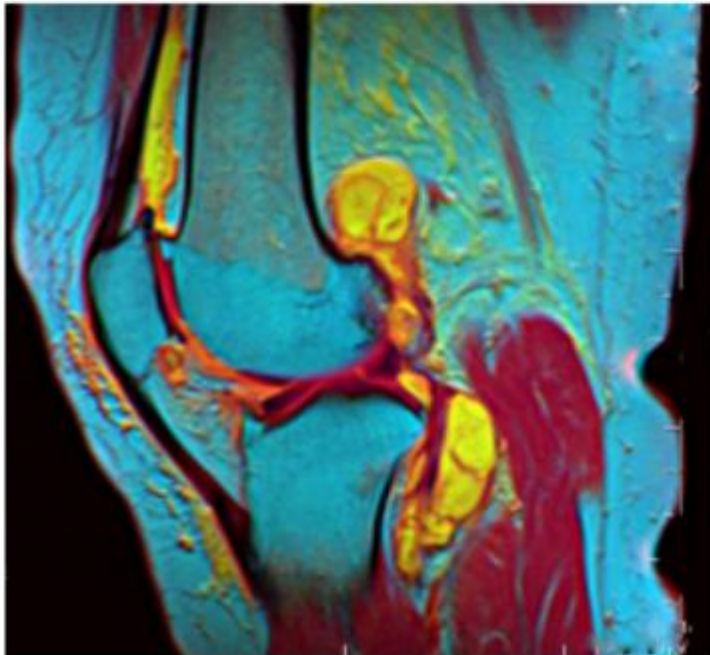
- The histogram of an RGB image can be displayed in terms of three separate histograms—one for each color component (R, G, and B) of the image.



Histogram in Python (using matplotlib)

```
1 from skimage import io
2 import matplotlib.pyplot as plt
3
4 image = io.imread('D:\Teaching\Pengola
5 print('dtype:', image.dtype)
6 print('shape:', image.shape)
7
8 plt.imshow(image)
9 plt.axis('off')
10 plt.show()
```

dtype: uint8
shape: (517, 560, 3)



```
1 hiscolour = plt.hist(image.ravel(), bins = 256, color = 'orange', alpha = 0.5 )
2 hiscolour = plt.hist(image[:, :, 0].ravel(), bins = 256, color = 'red', alpha = 0.5)
3 hiscolour = plt.hist(image[:, :, 1].ravel(), bins = 256, color = 'Green', alpha = 0.5)
4 hiscolour = plt.hist(image[:, :, 2].ravel(), bins = 256, color = 'Blue', alpha = 0.5)
5 hiscolour = plt.xlabel('Intensity Value')
6 hiscolour = plt.ylabel('Count')
7 hiscolour = plt.legend(['Total', 'Red_Channel', 'Green_Channel', 'Blue_Channel'])
8 plt.show()
```

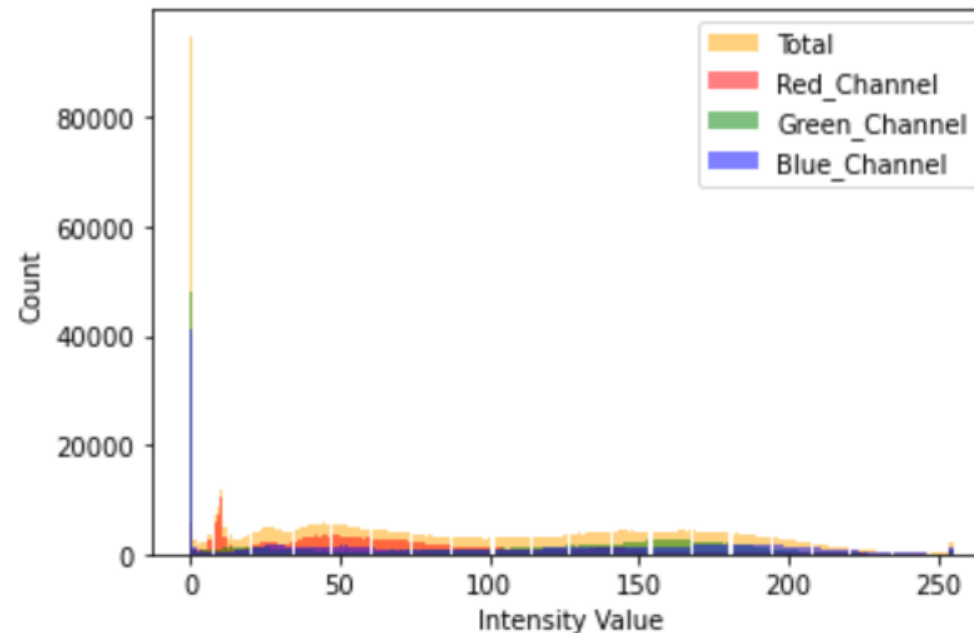


Image Brightness

- Brightness of a grayscale image is the **average intensity** of all pixels in image

$$B(I) = \frac{1}{wh} \sum_{v=1}^h \sum_{u=1}^w I(u, v)$$

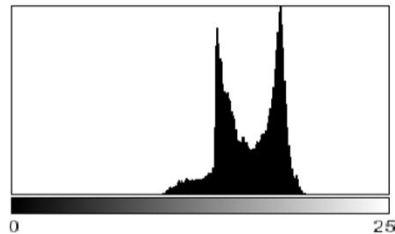
2. Divide by total number of pixels

1. Sum up all pixel intensities

Image Contrast

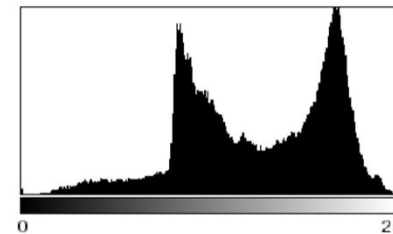
Good Contrast? Widely spread intensity values
+ large difference between min and max intensity values

- The contrast of a grayscale image indicates how easily objects in the image can be distinguished
- **High contrast image:** many distinct intensity values
- **Low contrast:** image uses few intensity values



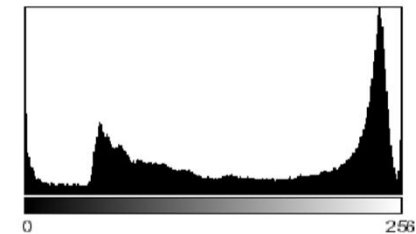
(a)

Low contrast



(b)

Normal contrast



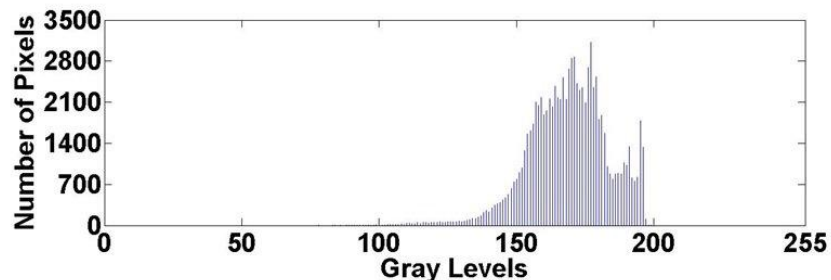
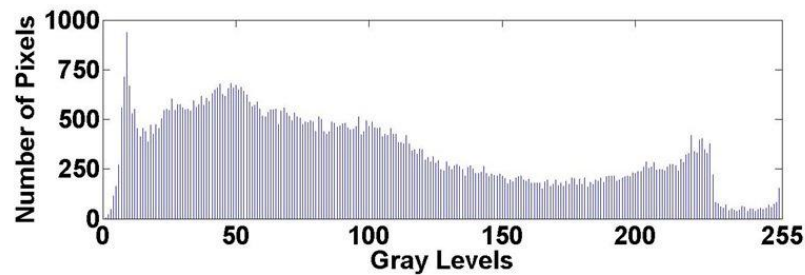
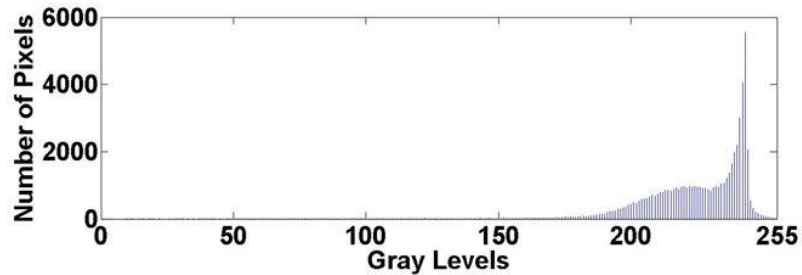
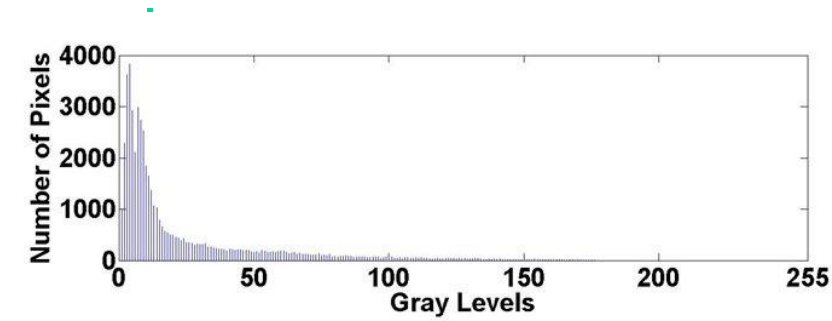
(c)

High contrast

Image

Histogram

Contrast and brightness of image



(b)

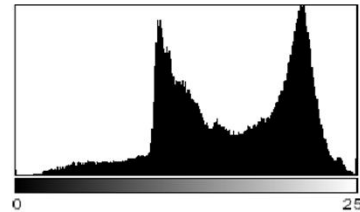
(a)

- Dark image: histograms are concentrated toward the left.
- Light image: histograms are concentrated toward the right.
- High contrast: histograms cover a broad range of grayscale level.
- Low contrast: Histograms are restricted to a smaller range

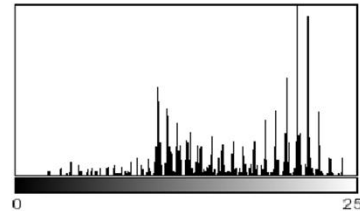
Image Defects: Effect of Image Compression

- Histograms show impact of image compression
- Example: in GIF compression, dynamic range is reduced to only few intensities (quantization)

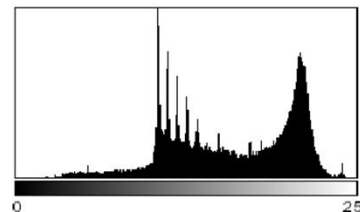
Original Image



(a) Original Histogram



(b) Histogram after GIF conversion



(c) Fix? Scaling image by 50% and Interpolating values recreates some lost colors

But GIF artifacts still visible

What Is Image Enhancement?

- Image enhancement makes images more useful by:
 - Highlighting interesting detail in images
 - Removing noise from images
 - Making images more visually appealing

Image Enhancement

Image enhancement is divided into several types:

- 1. Image enhancement in **the spatial domain**: The spatial domain method refers to the image plane itself and the approach is based on direct manipulation of the image pixels. Can be divided into two, namely:
 - (a) Point Processing
 - (b) Mask Processing
- 2. Image enhancement in **the frequency domain**: The frequency domain technique is based on the modification of the Fourier transform of an image.

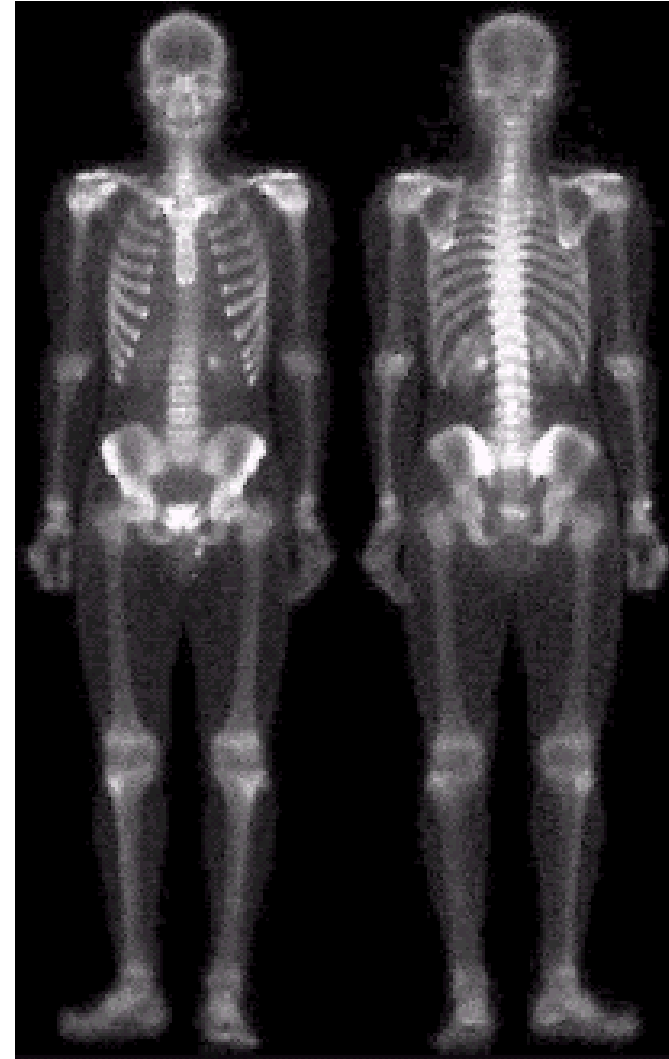
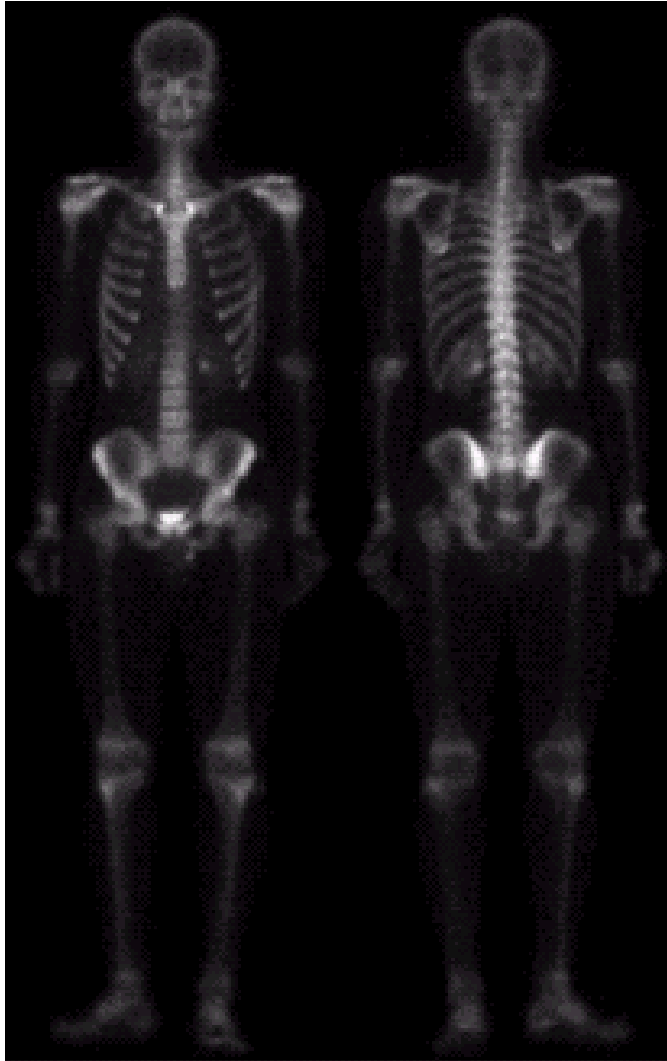
Image Enhancement Examples -1

Images taken from Gonzalez & Woods, Digital Image Processing (2002)



Image Enhancement Examples -2

Images taken from Gonzalez & Woods, Digital Image Processing (2008)





POINT PROCESSING

Point Operations

- Point operations changes a pixel's intensity value according to some function (don't care about pixel's neighbor)

$$a' \leftarrow f(a)$$

$$I'(u, v) \leftarrow f(I(u, v))$$

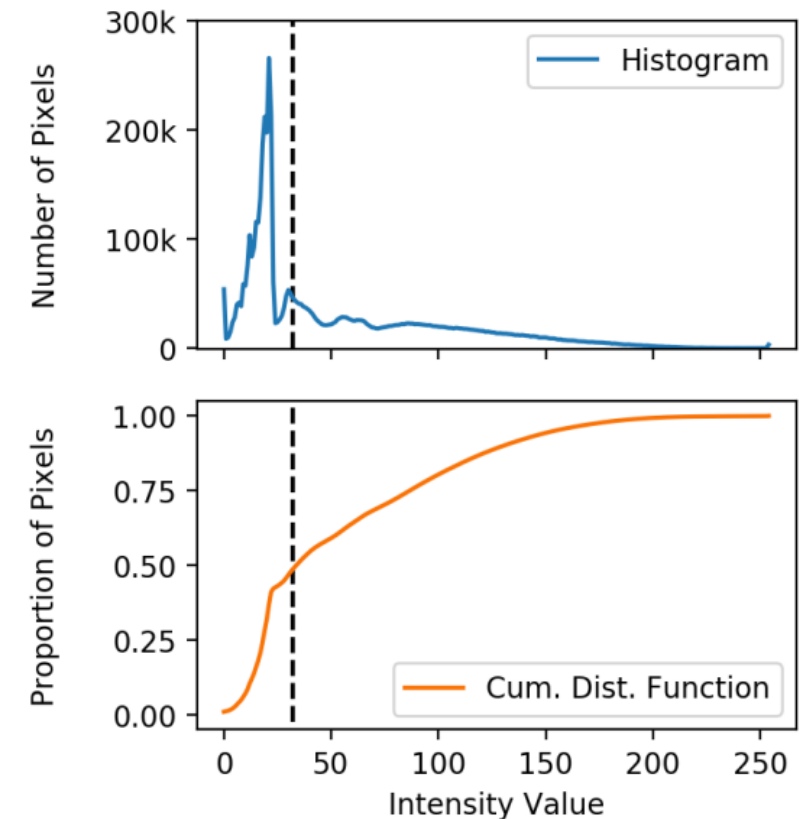
- Also called a **homogeneous operation**
- New pixel intensity **depends on**
 - Pixel's previous intensity $I(u, v)$
 - Mapping function $f()$
- **Does not depend on**
 - Pixel's location (u, v)
 - Intensities of neighboring pixels

Cumulative Histogram

- Useful for certain operations (e.g. histogram equalization).
- Analogous to the **Cumulative Density Function (CDF)**
- Definition:

$$s_k = T(r_k) = \sum_{j=0}^k \frac{n_j}{n} = \sum_{j=0}^k p(r_j), \quad 0 \leq r_k \leq 1, \quad k = 0, 1, \dots, L-1$$

L = maximal gray level



Cumulative Histogram (Calculation)

- Ex: An image have spatial resolution 4x5 and gray levels range of 0-10 (L=11). Find matrix of output image using Cumulative Density Function (CDF)!

Raw image:

3	5	5	5	4
5	4	5	4	4
5	3	4	4	4
4	5	6	6	3

← CDF

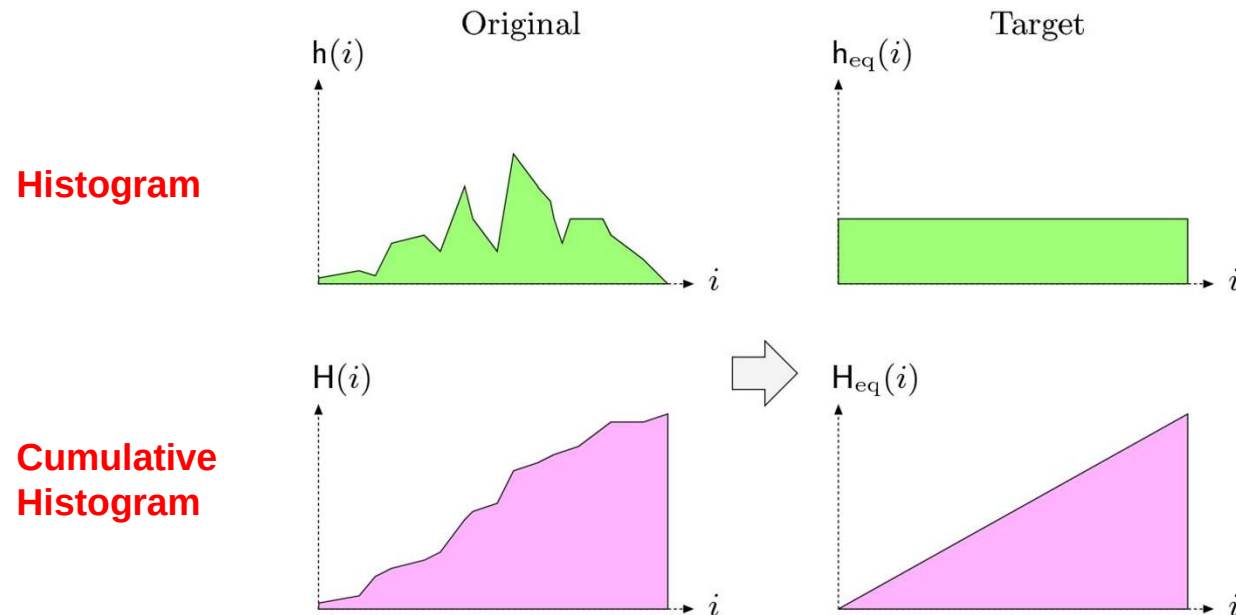
Grey-level input	0	1	2	3	4	5	6	7	8	9	10
Kemunculan	0	0	0	3	8	7	2	0	0	0	0
Probabilitas	0	0	0	0,15	0,4	0,35	0,1	0	0	0	0
Sk	0	0	0	0,15	0,55	0,9	1	1	1	1	1
Sk*10	0	0	0	1,5	5,5	9	10	10	10	10	10
Grey-level output	0	0	0	1	5	9	10	10	10	10	10

Output image:

1	9	9	9	5
9	5	9	5	5
9	1	5	5	5
5	9	10	10	1

Histogram Equalization

- Adjust 2 different images to make their histograms (intensity distributions) similar
- Apply a point operation that changes histogram of modified image into **uniform distribution**



Histogram Equalization-1 (Python)

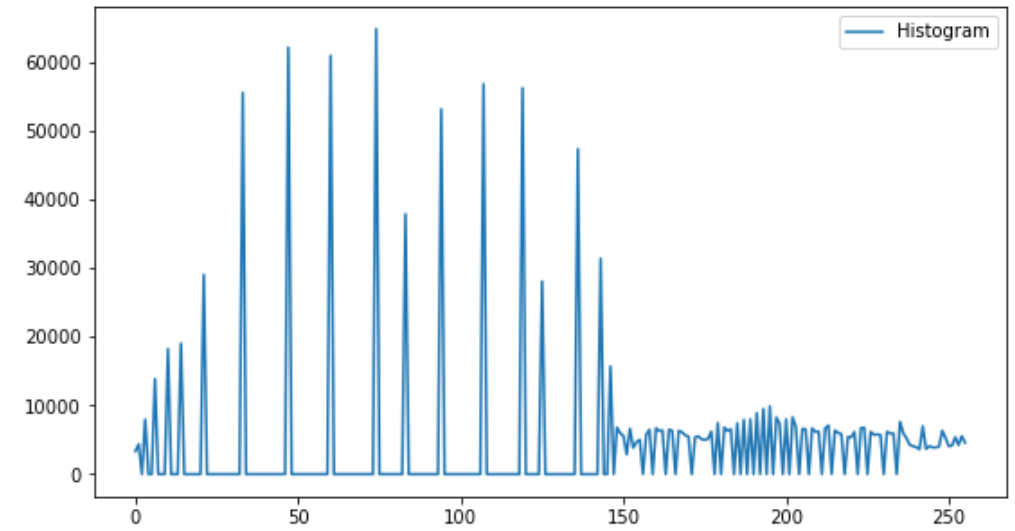
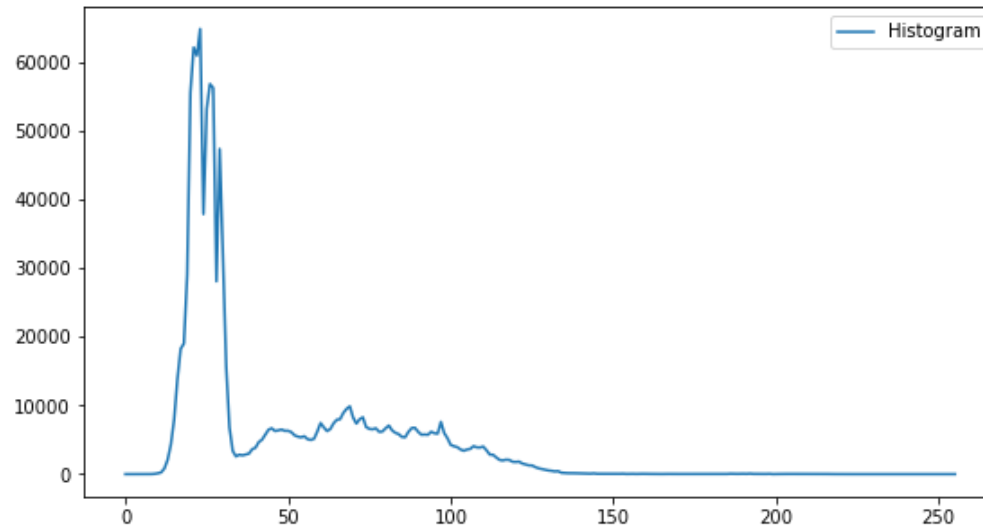
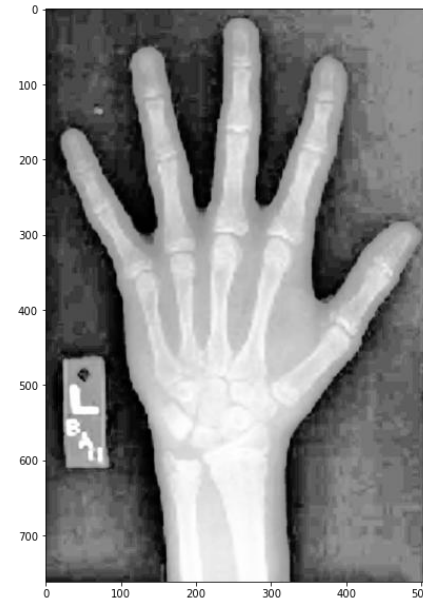
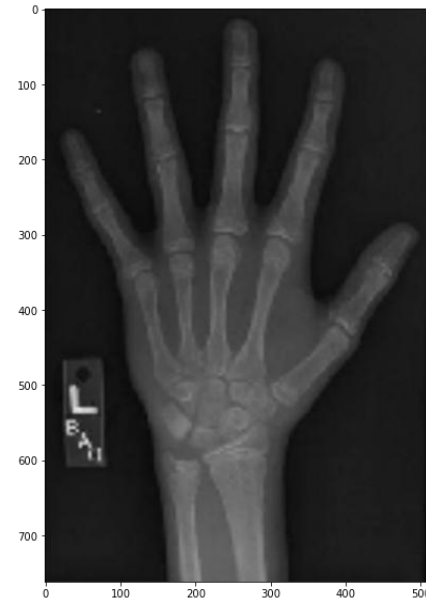
```
1 import numpy as np
2 import scipy.ndimage as ndi
3
4 im_equalized = cdf[im] * 255
5
6 #-----Untuk membenarkan clipping---#
7 im_equalized = im_equalized/np.amax(im_equalized) #
8 im_equalized = np.clip(im_equalized, 0, 255) #
9
10 #-----Tampilkan-----#
11 fig, axes = plt.subplots(1, 2, figsize=(20, 10)) #mer
12 axes[0].imshow(im)
13 axes[1].imshow(im_equalized)
14
15 plt.show()
```

Jika menggunakan skimage, silahkan cek programnya disini:

https://scikit-image.org/docs/0.9.x/auto_examples/plot_equalize.html

```
1 hist_equalized = ndi.histogram(im_equalized, min=0, max=255, bins=256)
2
3 # Plot the histogram and Equalization
4 fig, axes = plt.subplots(1, 2,figsize=(20, 5))
5 axes[0].plot(hist, label='Histogram')
6 axes[0].legend()
7 axes[1].plot(hist_equalized, label='Histogram')
8 axes[1].legend();
```

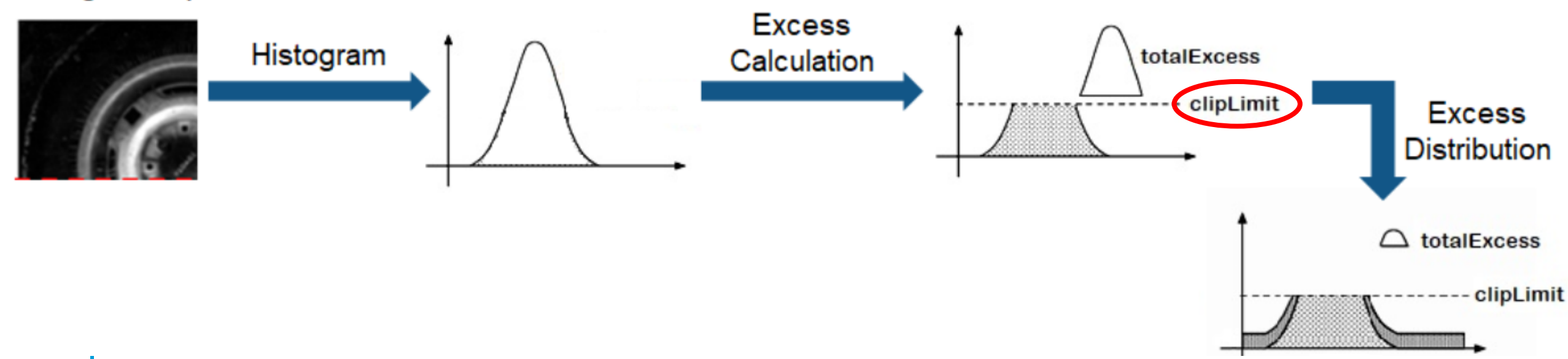
Histogram Equalization-2 (Python)



Adaptive Histogram Equalization (1)

- Adaptive histogram equalization (AHE) is an image pre-processing technique used to improve contrast in images. It computes several histograms, each corresponding to a distinct section of the image, and uses them to redistribute the luminance values of the image.
- AHE is suitable for improving the local contrast and enhancing the definitions of edges in each region of an image.

Histogram Equalization



Adaptive Histogram Equalization (2)

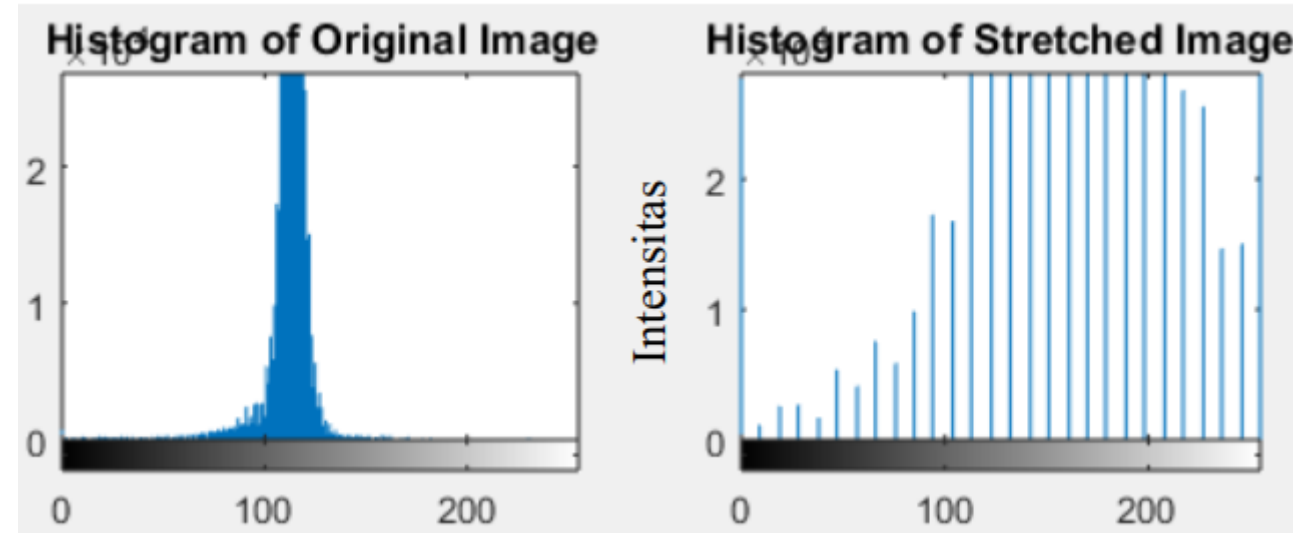
- We can use `skimage` and import `exposure` module.

```
1 #Adaptive Histogram Equalization
2 from skimage import exposure
3
4 img_adapteq = exposure.equalize_adapthist(im, clip_limit=0.03)
5 plt.imshow(img_adapteq, cmap='gray')
6 plt.show()
```



Contrast Stretching (1)

- Contrast stretching or normalization is a technique for improving image quality by stretching the intensity range at certain values (min and max).



$$f_{ac}(a) = a_{\min} + (a - a_{\text{low}}) \cdot \frac{a_{\max} - a_{\min}}{a_{\text{high}} - a_{\text{low}}}$$

If $a_{\min} = 0$ and $a_{\max} = 255$

$$f_{ac}(a) = (a - a_{\text{low}}) \cdot \frac{255}{a_{\text{high}} - a_{\text{low}}}$$

Contrast Stretching (2)

```
1 c = 8#im.min()    #Pin = im
2 d = 150#im.max()
3 a = 0
4 b = 255
5
6 stretched_image = (im - c) * ((b - a) / (d - c)) + a
7
8 plt.imshow(stretched_image)
9 plt.show()
10
11 hist_cs = ndi.histogram(stretched_image, min=0, max=255, bins=256)
12 plt.plot(hist_cs)
13 plt.show()
```

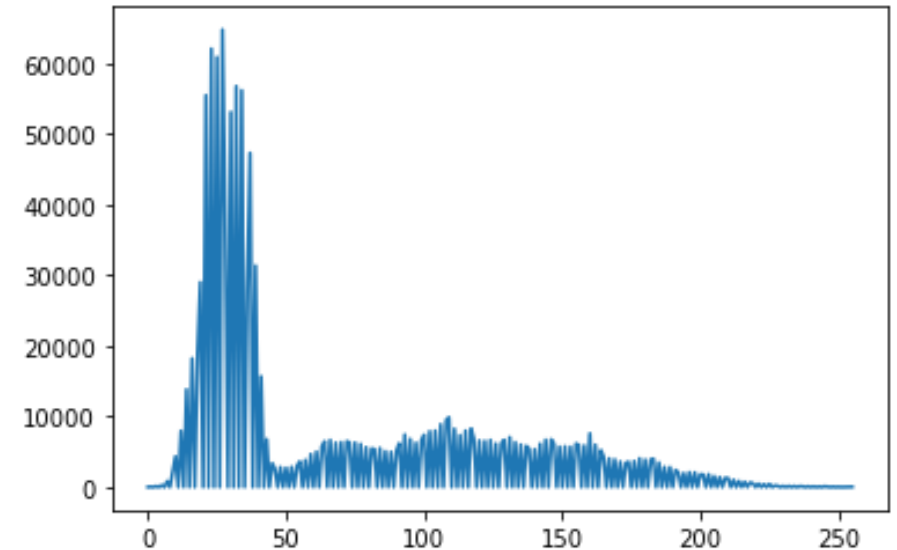
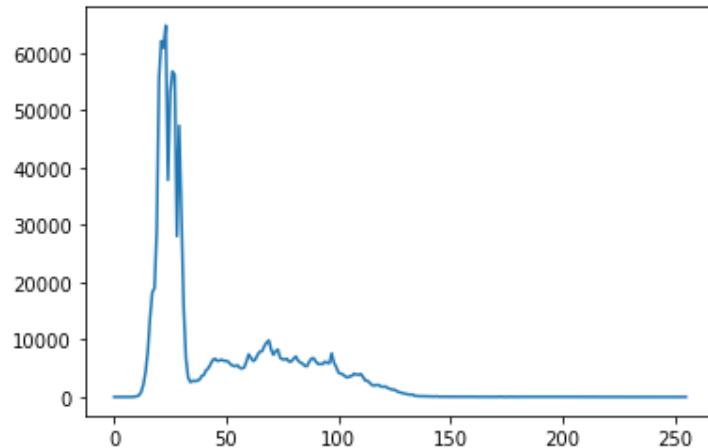
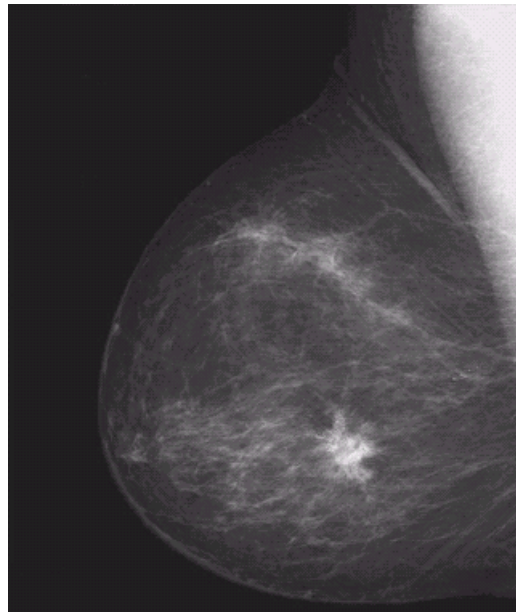


Image Negatives (Inverted Images)

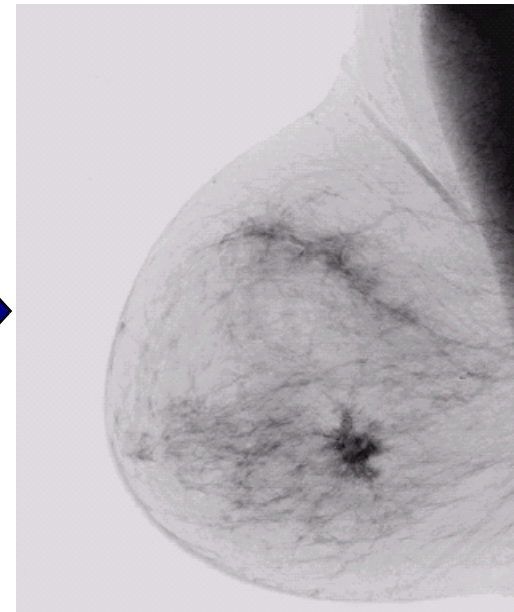
$$f_{\text{invert}}(a) = -a + a_{\text{max}} = a_{\text{max}} - a$$

- Image negatives useful for enhancing white or grey detail embedded in dark regions of an image
 - Note how much clearer the tissue is in the negative image of the mammogram below
- 2 steps
 1. Multiple intensity by -1
 2. Add constant (e.g. a_{max}) to put result in range $[0, a_{\text{max}}]$

Original
Image



$$s = 1.0 - r$$



Negative
Image

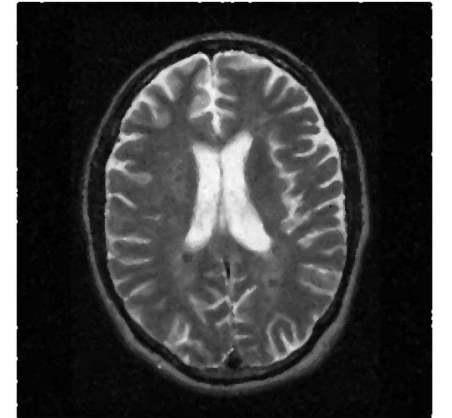


IMAGE FILTERING

NADA FITRIEYATUL HIKMAH

What is a Filter?

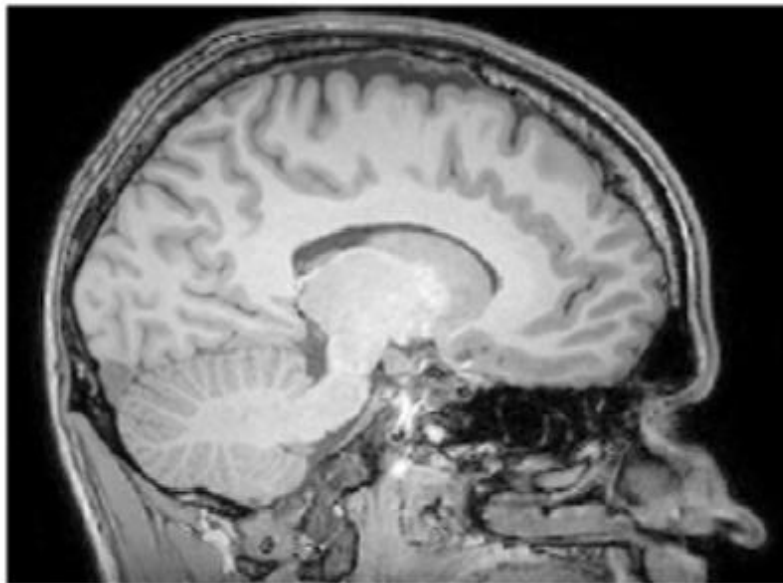
- Capabilities of point operations are limited
- **Filters:** combine **pixel's value** + **values of neighbors**
- **E.g blurring:** Compute average intensity of block of pixels



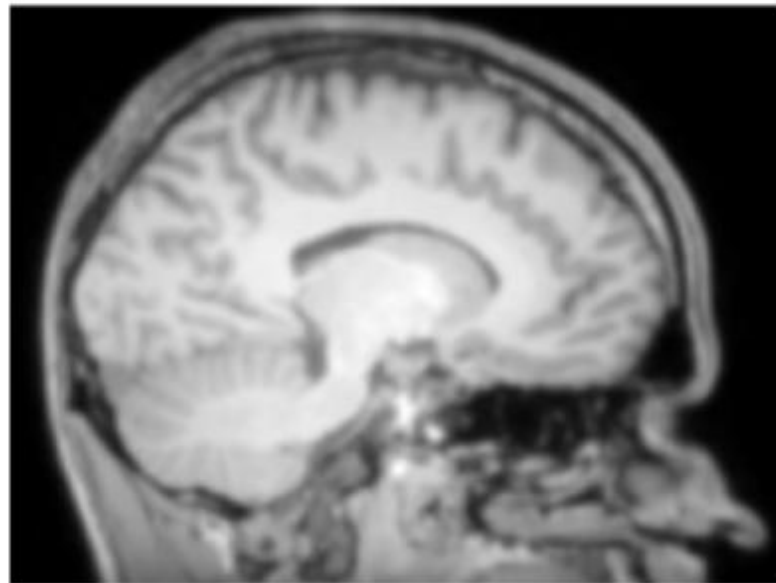
- Combining multiple pixels needed for certain operations:
 - Blurring, Smoothing
 - Sharpening

Filters

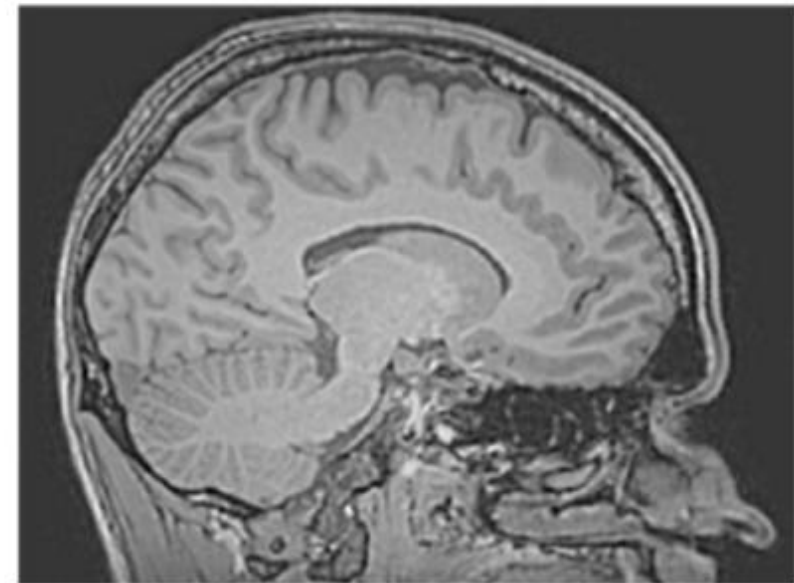
Original



Smoothed

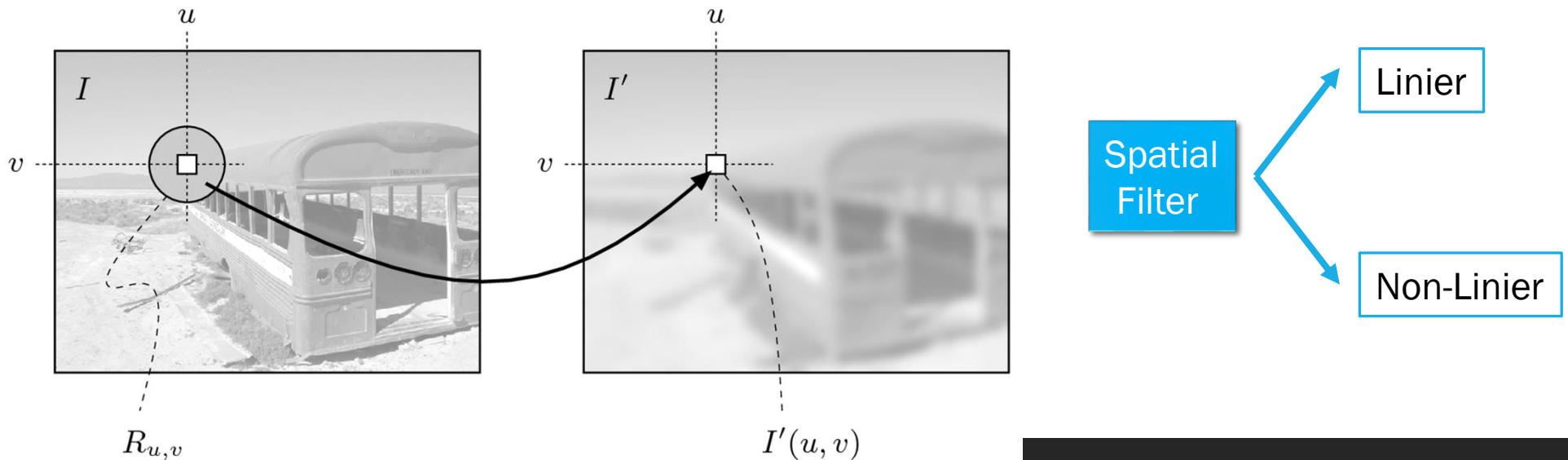


Sharpened



Spatial Filter

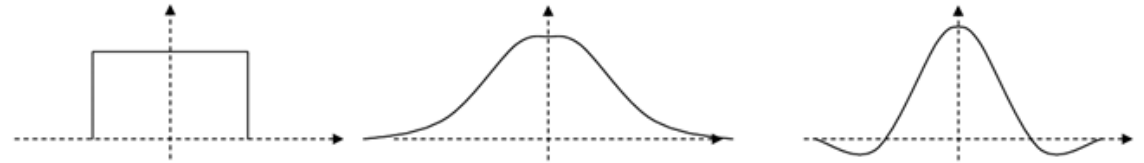
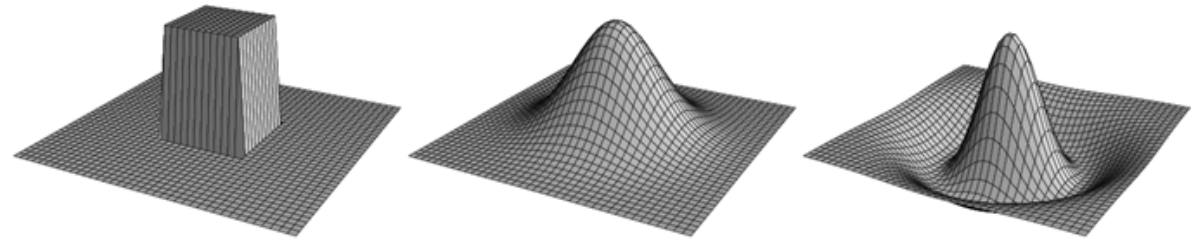
- An image operation that combines each pixel's intensity $I(u, v)$ with that of neighboring pixels
- **E.g:** average/weighted average of group of pixels



LINIER FILTER

Linear Filters: Smoothing Filters

- 2 main classes of linear filters:
 - **Smoothing:** +ve coefficients (weighted average). E.g box, gaussian
 - **Difference** filters: +ve and -ve weights. E.g. Laplacian



0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

(a)

Box

0	1	2	1	0
1	3	5	3	1
2	5	9	5	2
1	3	5	3	1
0	1	2	1	0

(b)

Gaussian

0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

(c)

Laplacian

The Filter Matrix (Kernel)

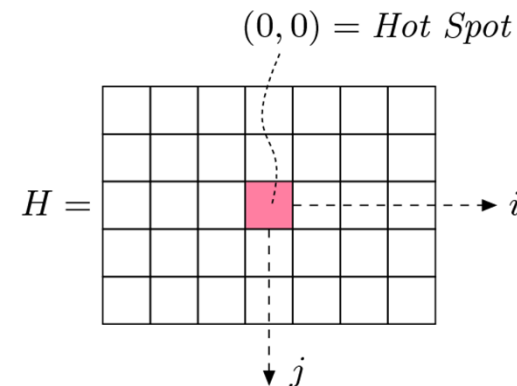
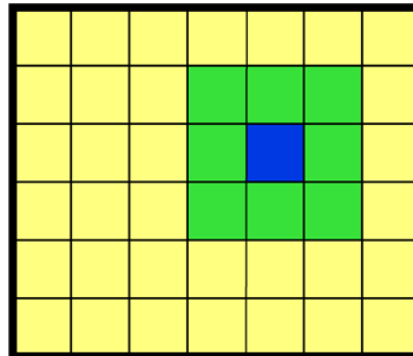
$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$H(i, j) = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Filter operation can be expressed as a matrix Example: averaging filter or Mean Filter

Filter matrix also called filter mask $H(i, j)$



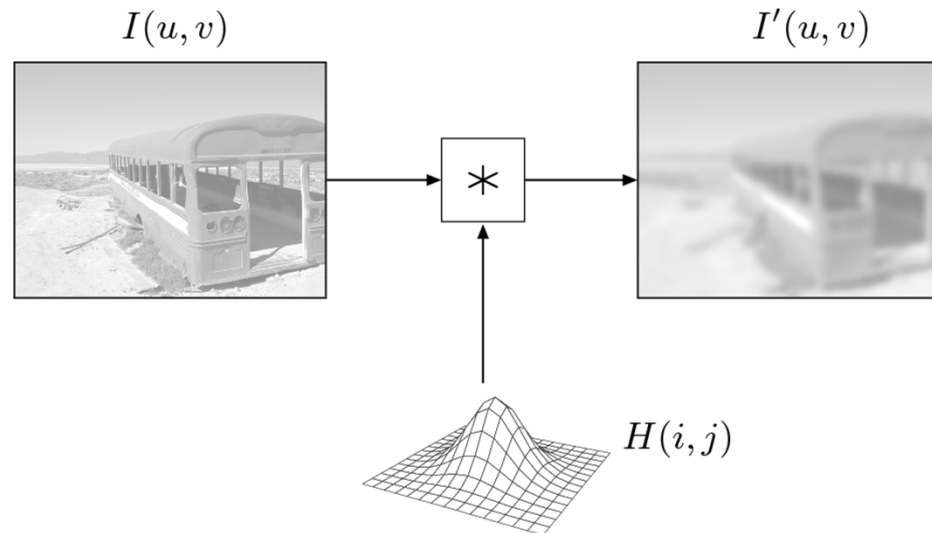
Convolution (1)

- For discrete 2D signal, convolution defined as:

$$I'(u, v) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} I(u-i, v-j) \cdot H(i, j)$$

Formal definition:
Sum to $\pm \infty$

$$I' = I * H$$



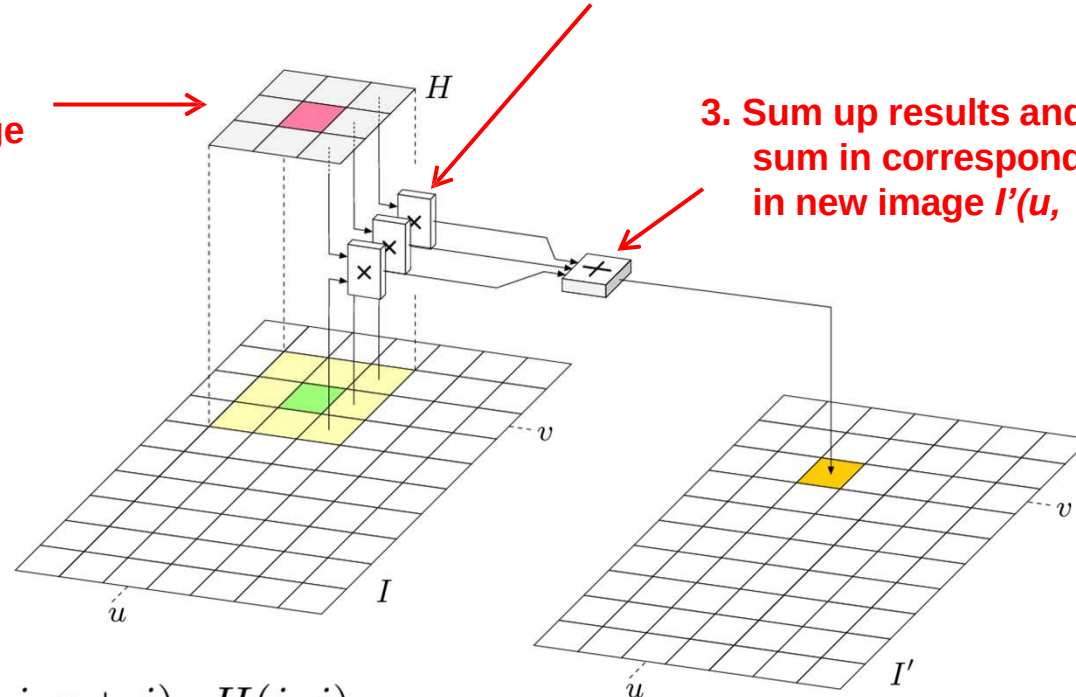
Convolution (2)

For each image position $I(u,v)$:

1. Move filter matrix H over image such that $H(0,0)$ coincides with current image position (u,v)

2. Multiply all filter coefficients $H(i,j)$ with corresponding pixel $I(u+i, v+j)$

3. Sum up results and store sum in corresponding position in new image $I'(u, v)$



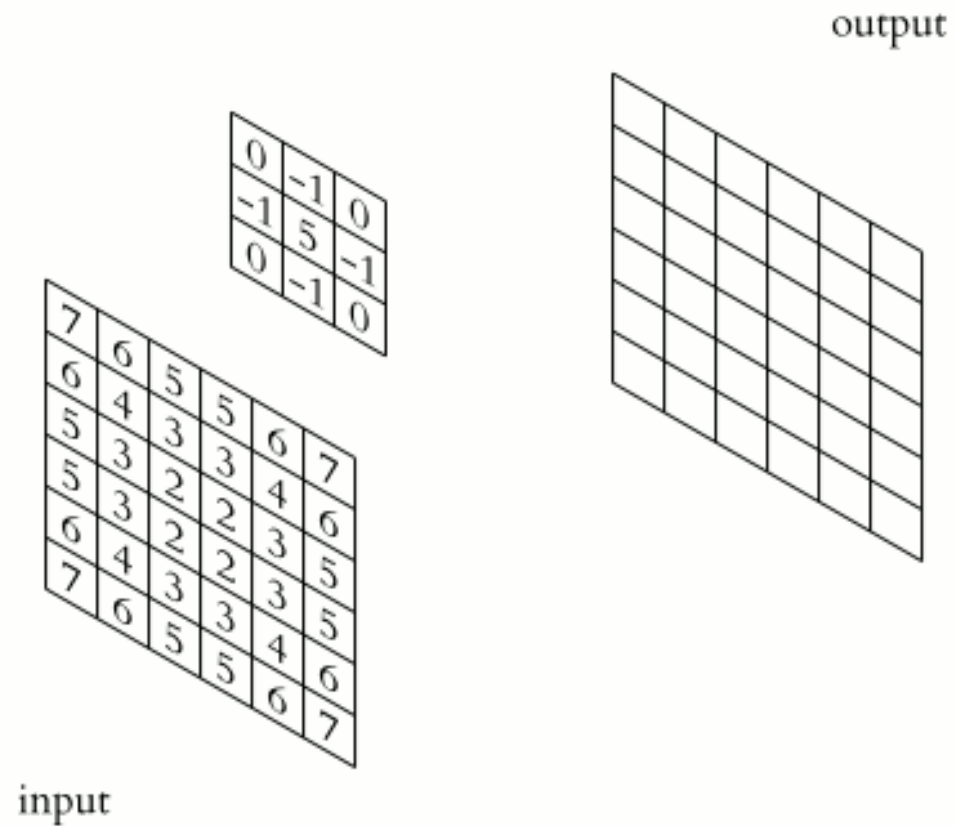
Stated formally:

$$I'(u, v) \leftarrow \sum_{(i,j) \in R_H} I(u+i, v+j) \cdot H(i, j)$$

R_H is set of all pixels covered by filter.
For 3x3 filter, this is:

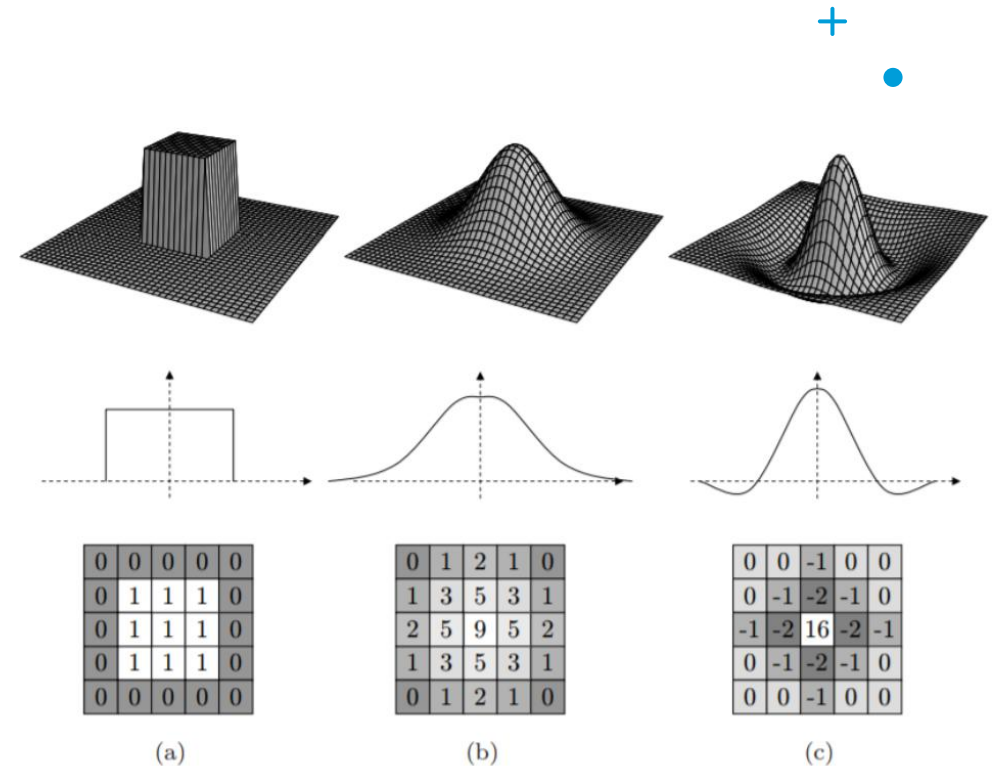
$$I'(u, v) \leftarrow \sum_{i=-1}^{i=1} \sum_{j=-1}^{j=1} I(u+i, v+j) \cdot H(i, j)$$

Convolution (3)



Type of Linear Filter

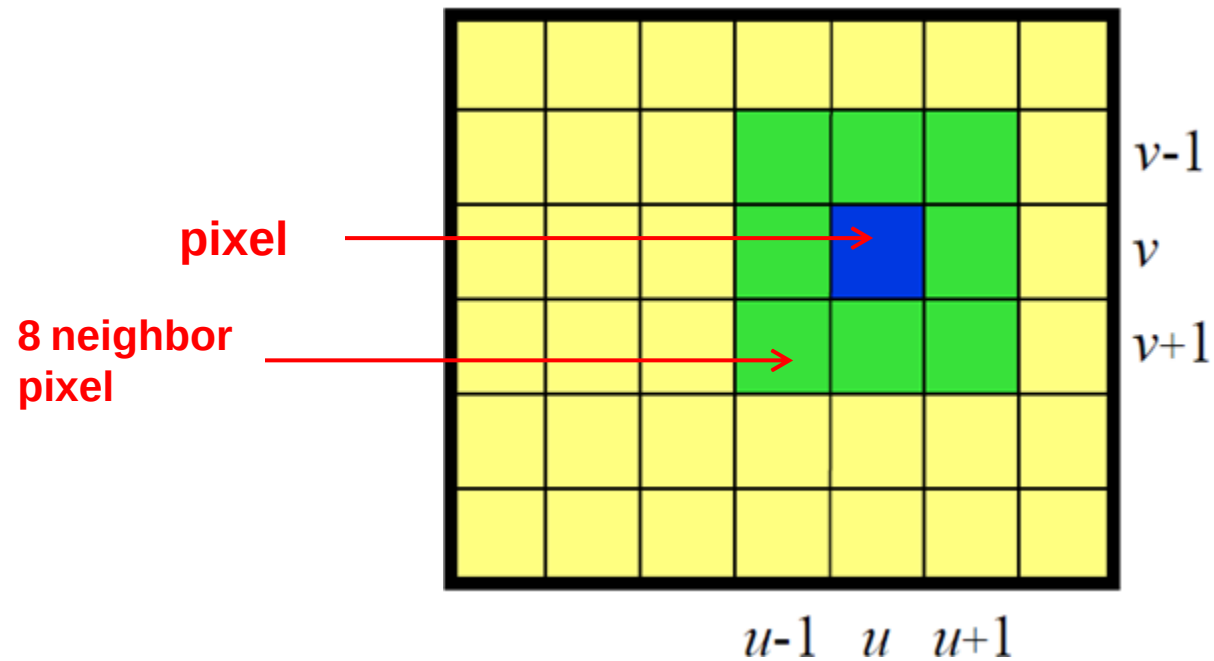
1. **Smoothing Filter** (This filter has only positive integer.)
 - **Box filter.** All members of this filter are the same.
 - **Gaussian filter.** The weight of filter member depend on the location of the member. The center of the filter receive the maximum weigh and it decreases with distance from the center.
2. **Difference Filter**
 - **Laplace or Mexican hat filter.** Some members of this filter are negative filters, and it can calculate by summation of positive member and negative member.



3D structure, 2D structure and example of filter (a) Box filter (b) Gaussian filter and (c) Laplace filter

Mean Filter (1)

Example: Average (Mean) of 3x3 Neighborhood

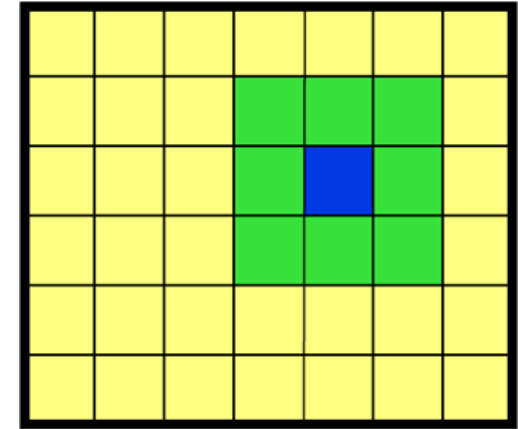
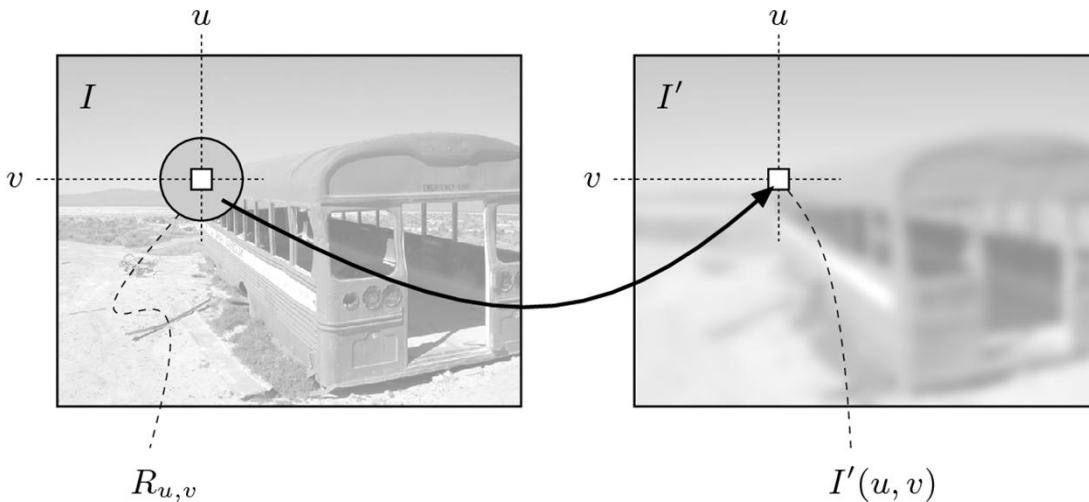


Blurring: Replace each pixel with AVERAGE Intensity of pixel + neighbors

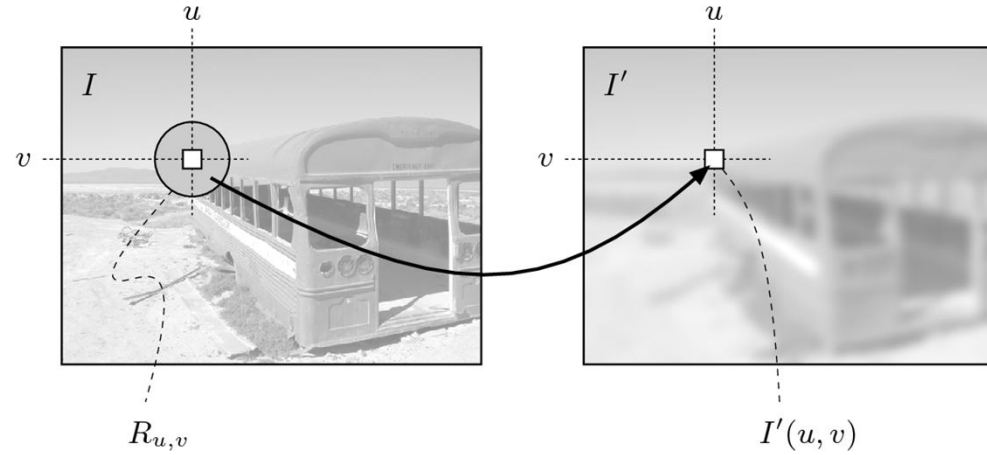
Mean Filter (2)

- Replace each pixel by average of pixel + neighbors
- For 3x3 neighborhood:

$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$



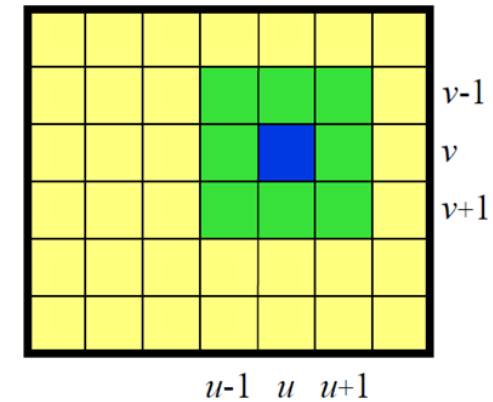
Mean Filter (3)



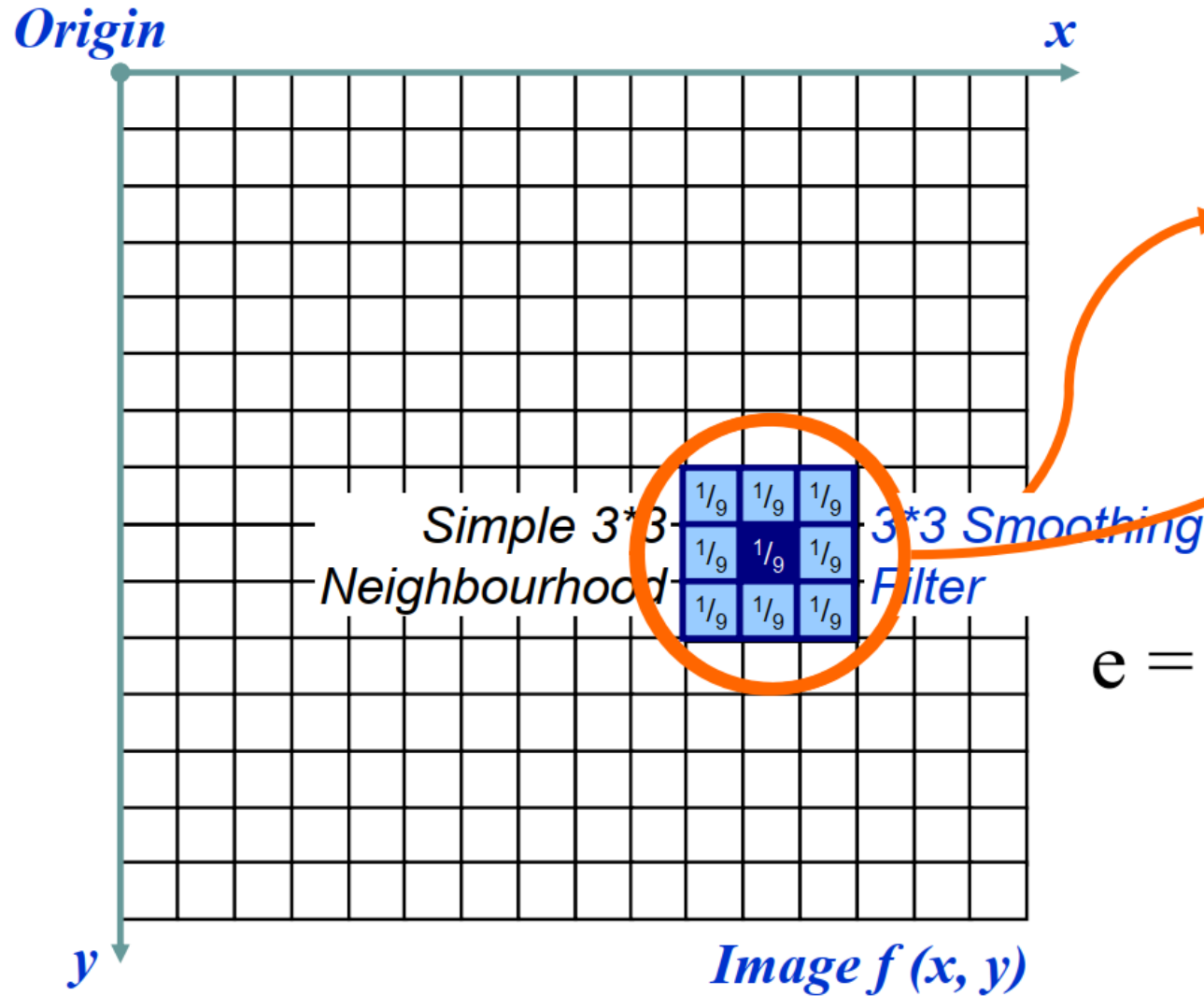
$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot \sum_{j=-1}^1 \sum_{i=-1}^1 I(u+i, v+j)$$



Mean Filter (4)



104	100	108
99	106	98
95	90	85

**Original Image
Pixels**



$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$

Filter

$$\begin{aligned} e &= \frac{1}{9} * 106 + \\ &\quad \frac{1}{9} * 104 + \frac{1}{9} * 100 + \frac{1}{9} * 108 + \\ &\quad \frac{1}{9} * 99 + \frac{1}{9} * 98 + \\ &\quad \frac{1}{9} * 95 + \frac{1}{9} * 90 + \frac{1}{9} * 85 \\ &= 98.3333 \end{aligned}$$

Mean Filter (Python)

```
1 import imageio
2 import numpy as np
3 import scipy.ndimage as ndi
4 import matplotlib.pyplot as plt
5
6 # Load the hand radiograph
7 im = imageio.imread('D:\Teaching\Pengolahan Citra Medika\Database\hand.png')
8 weights = np.array([[0.11,0.11,0.11],[0.11,0.11,0.11],[0.11,0.11,0.11]])
9 weights
```

```
array([[0.11, 0.11, 0.11],
       [0.11, 0.11, 0.11],
       [0.11, 0.11, 0.11]])
```

```
1 #cara singkat state kernel atau weight
2 weights = np.full((3, 3), 1/9)
3 weights
```

```
array([[0.11111111, 0.11111111, 0.11111111],
       [0.11111111, 0.11111111, 0.11111111],
       [0.11111111, 0.11111111, 0.11111111]])
```

```
1 im_filt = ndi.convolve(im,weights)
2 fig, axes = plt.subplots(2, 1, figsize=(20,10))
3 axes[0].imshow(im, cmap='gray')
4 axes[1].imshow(im_filt, cmap='gray')
5 plt.show()
```



Mean Filter from Scratch

$$I'(u, v) \leftarrow \frac{1}{9} \cdot \sum_{j=-1}^1 \sum_{i=-1}^1 I(u+i, v+j)$$

```
def MeanFilter(image, filter_size):
    # create an empty array with same size as input image
    output = np.zeros(image.shape, np.uint8)

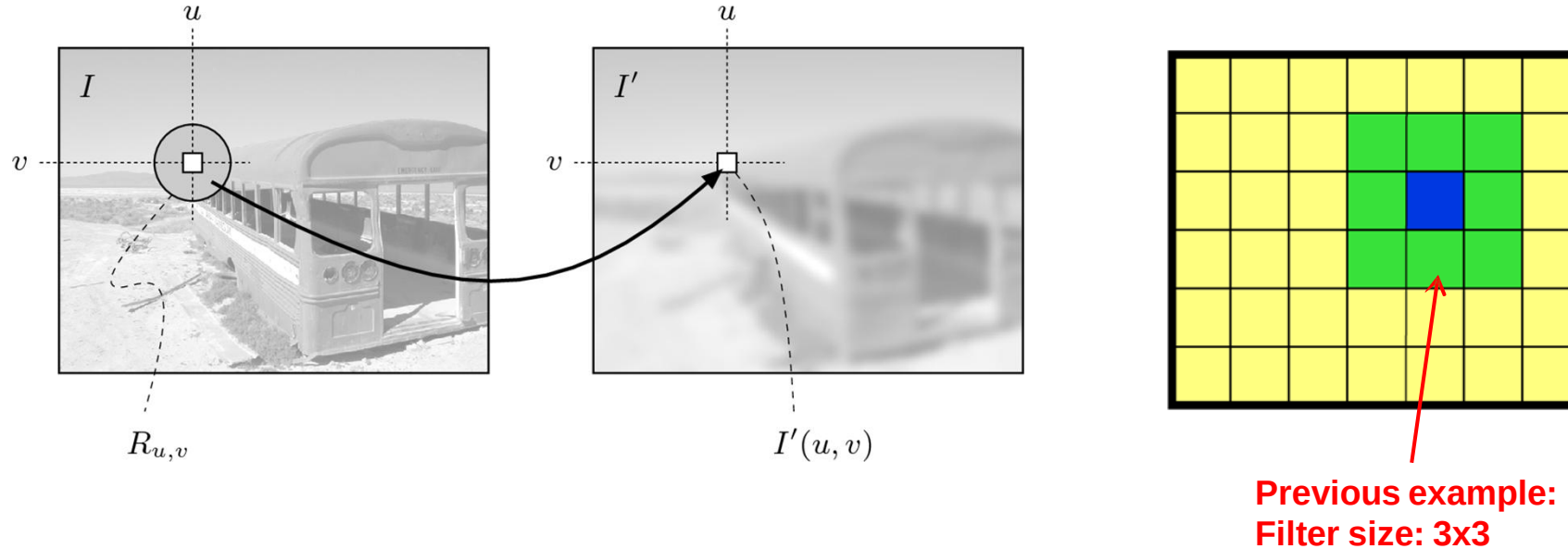
    # creat an empty variable
    result = 0

    # filter size = 3x3
    if filter_size == 9:
        for j in range(1, image.shape[0]-1):
            for i in range(1, image.shape[1]-1):
                for y in range(-1, 2):
                    for x in range(-1, 2):
                        result = result + image[j+y, i+x]
                    output[j][i] = int(result / filter_size)
                result = 0

    # filter size = 5x5
    elif filter_size == 25:
        for j in range(2, image.shape[0]-2):
            for i in range(2, image.shape[1]-2):
                for y in range(-2, 3):
                    for x in range(-2, 3):
                        result = result + image[j+y, i+x]
                    output[j][i] = int(result / filter_size)
                result = 0

    return output
```

Smoothing an Image by Averaging



- Many possible filter parameters (size, weights, function, etc)
- **Filter size (size of neighborhood):** 3x3, 5x5, 7x7, ..., 21x21, ..
- **Filter shape:** not necessarily square. Can be rectangle, circle, etc
- **Filter weights:** May apply unequal weighting to different pixels
- **Filters function:** can be linear (a weighted summation) or nonlinear

Mean Filters: Effect of Filter Size



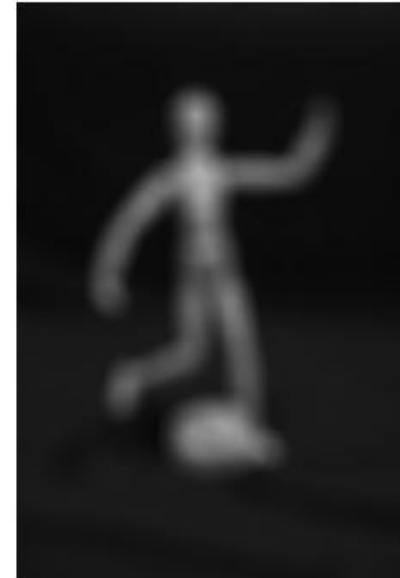
Original



7×7



15×15



41×41

Weighted Smoothing Filters

- More effective smoothing filters can be generated by allowing different pixels in the neighborhood different weights in the averaging function

- Pixels closer to central pixel more important
- Often referred to as a *weighted averaging*

$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$
$\frac{2}{16}$	$\frac{4}{16}$	$\frac{2}{16}$
$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$

Weighted averaging filter

Gaussian Filter

- Smoothing can improve the signal-to-noise ratio of your image by blurring out **small variations in intensity**. The Gaussian filter is excellent for this!
- The width of the distribution is controlled by the **sigma** argument, with higher values leading to larger smoothing effect.
- Gaussian kernel:

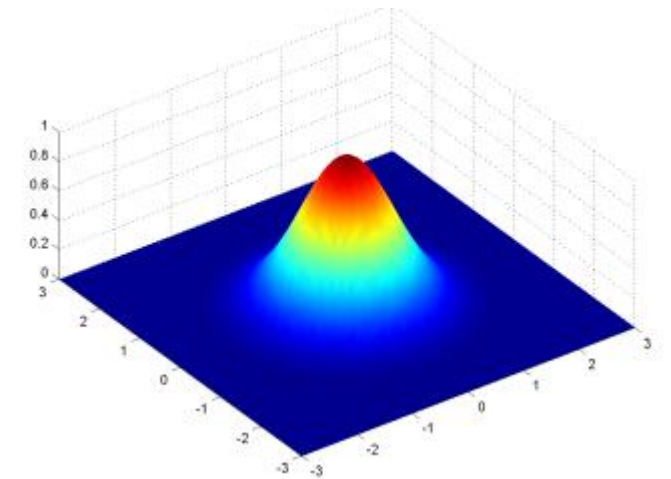
- 1D

$$g_{\sigma}(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right)$$



- 2D

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

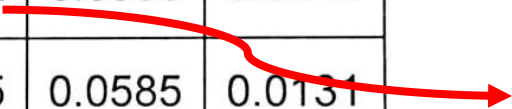


Gaussian Filter - Calculation

- Kernel distribusi Gaussian 2-D dengan sigma = 1 dan ukuran kernel 5 x 5 :

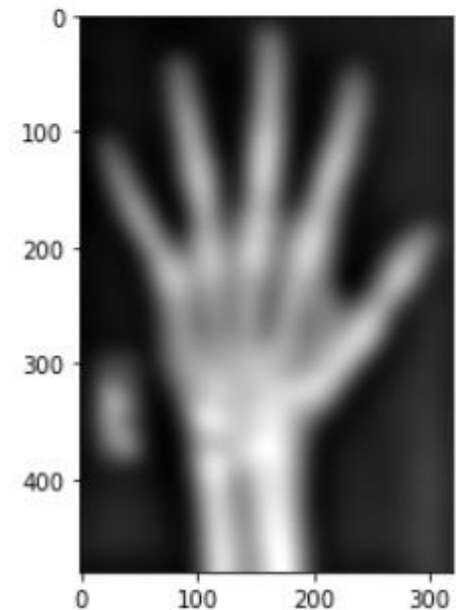
0.0029	0.0131	0.0215	0.0131	0.0029
0.0131	0.0585	0.0965	0.0585	0.0131
0.0215	0.0965	0.1592	0.0965	0.0215
0.0131	0.0585	0.0965	0.0585	0.0131
0.0029	0.0131	0.0215	0.0131	0.0029

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$


$$G(0,0) = \frac{1}{\sqrt{(2).(3.14).(1^2)}} \exp\left(-\frac{(0^2 + 0^2)}{2.(1^2)}\right) = 0.1592$$

Gaussian Filter – Python

```
1 im_Gauss1 = ndi.gaussian_filter(img_adapteq, sigma = 2)
2 im_Gauss2 = ndi.gaussian_filter(img_adapteq, sigma = 10)
3
4 fig, axes = plt.subplots(2, 1, figsize=(20,10))
5 axes[0].imshow(im_Gauss1, cmap='gray')
6 axes[1].imshow(im_Gauss2, cmap='gray')
7 plt.show()
```



NOISE

Noise

- While taking picture (during capture), noise may occur
- Noise? Errors, degradations in pixel values
- Examples of causes:
 - Focus blurring
 - Blurring due to camera motion
- Additive model for noise:
$$H * I + \text{Noise}$$
- Removing noise called **Image Restoration**
- Image restoration can be done in:
 - Spatial domain, or
 - Frequency domain

Types of Noise (1)

- Type of noise determines best types of filters for removing it!!
- **Salt and pepper noise:** Randomly scattered black + white pixels
- Also called **impulse noise, shot noise or binary noise**
- Caused by sudden sharp disturbance



(a) Original image

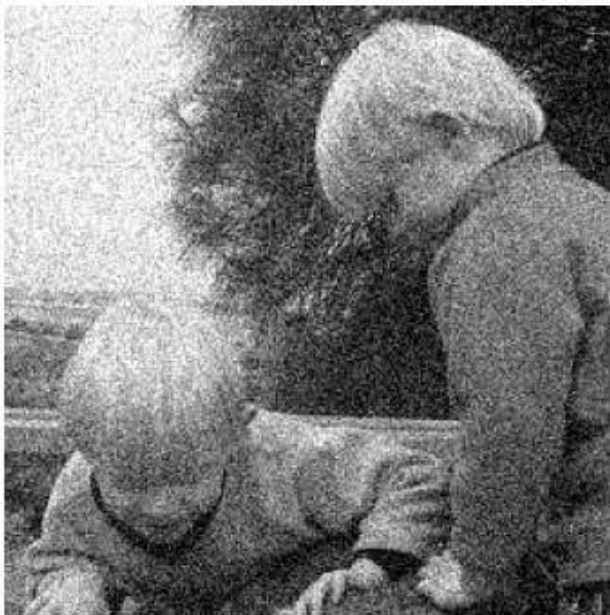


(b) With added salt & pepper noise

*Courtesy
Allasdair McAndrews*

Types of Noise (2)

- **Gaussian Noise:** idealized form of white noise *added to* image, normally distributed $I + \text{Noise}$
- **Speckle Noise:** pixel values *multiplied by* random noise $I(1 + \text{Noise})$



(a) Gaussian noise



(b) Speckle noise

Courtesy
Allasdair McAndrews

Types of Noise (3)

- **Periodic Noise:** caused by disturbances of a periodic nature
- Salt and pepper, gaussian and speckle noise can be cleaned using spatial filters
- Periodic noise can be cleaned using frequency domain filtering

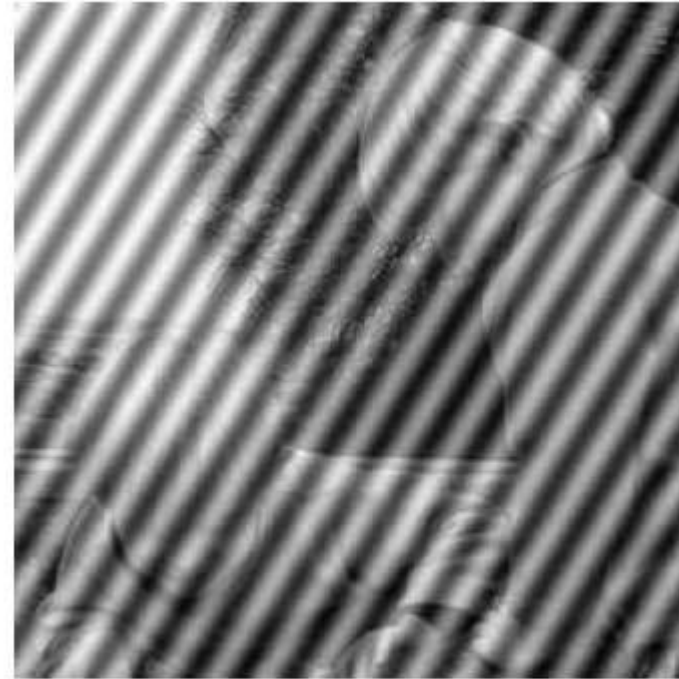


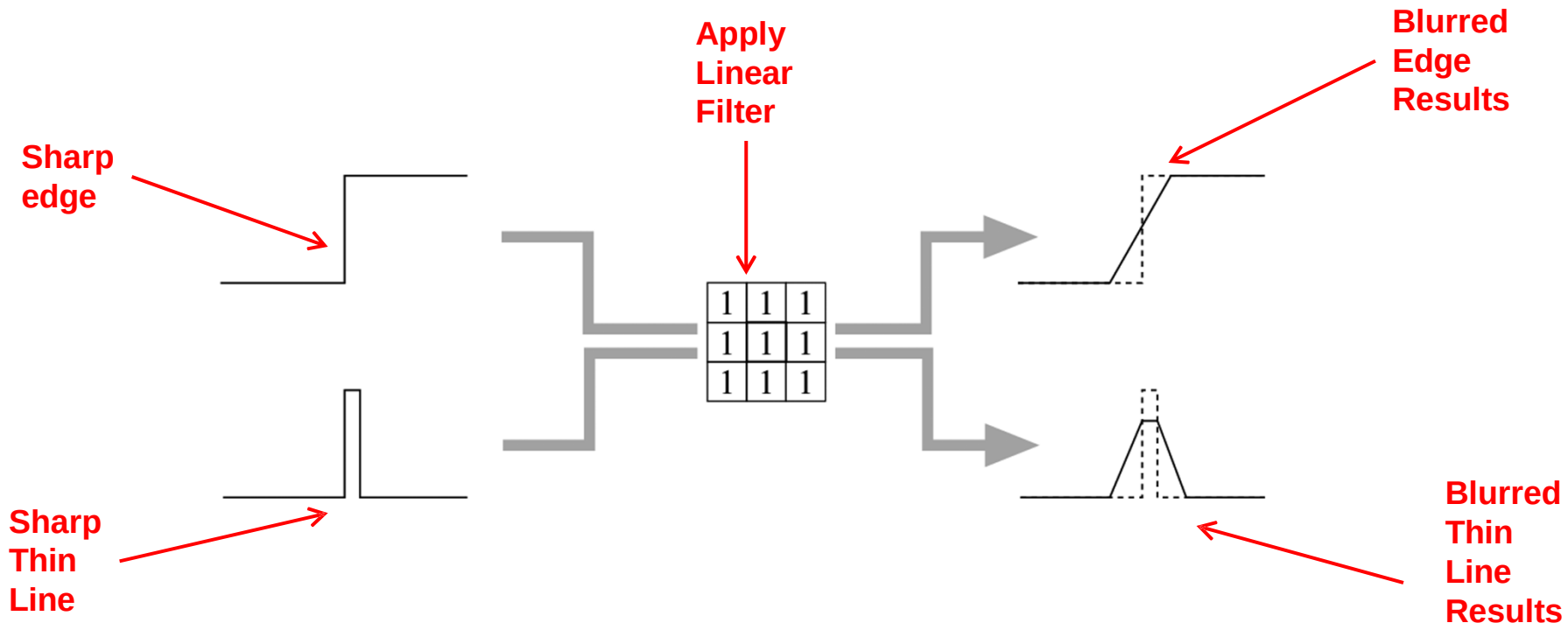
Figure 5.3: The twins image corrupted by periodic noise

*Courtesy
Allasdair McAndrews*

NON-LINIER FILTER

Non-Linear Filters

- Linear filters blurs all image structures points, edges and lines, reduction of image quality (**bad!**)
- Linear filters thus not used a lot for removing noise



Using Linear Filter to Remove Noise?

- **Example:** Using linear filter to clean salt and pepper noise just causes smearing (not clean removal)
- Try non-linear filters?

*Courtesy
Allasdair McAndrews*



(a) 3×3 averaging



(b) 7×7 averaging

Non-Linear Filters

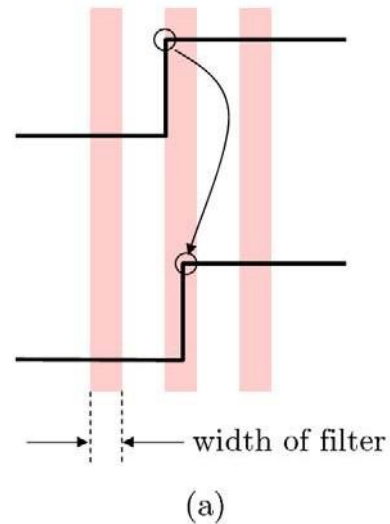
- Pixels in filter range combined by some non-linear function
- Simplest examples of nonlinear filters: **Min and Max filters**

$$I'(u, v) \leftarrow \min \{I(u+i, v+j) \mid (i, j) \in R\}$$

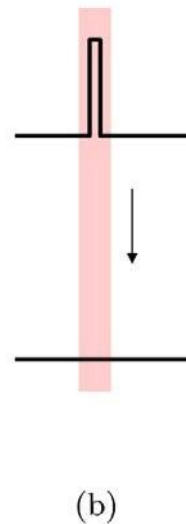
$$I'(u, v) \leftarrow \max \{I(u+i, v+j) \mid (i, j) \in R\}$$

Before
filtering

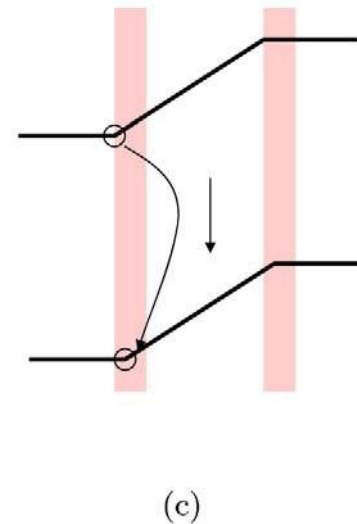
After
filtering



Step Edge
(shifted to right)



Narrow
Pulse
(removed)



Linear Ramp
(shifted to right)

Effect of
Minimum
filter

FORMULA:

Max Filter

$$\hat{f}(x, y) = \max_{(s, t) \in S_{xy}} \{g(s, t)\}$$

- Max Filter mengganti nilai sebuah piksel dengan nilai terbesar dari gray-level dalam subimage.
- Max Filter berfungsi untuk mencari titik paling terang dalam citra.
- Max Filter baik untuk menghilangkan **pepper noise** yang memiliki nilai gray-level noise yang rendah.

Kernel : 3x3

Citra *input* sebagai berikut :

5	6	5	7	8
9	10	1	3	2
5	6	10	3	2
7	4	4	5	1
2	3	5	3	3

Maka hasil *max filtering* adalah :

5	6	5	7	8
9	10	10	10	2
5	10	10	10	2
7	10	10	10	1
2	3	5	3	3

$F(2,2) = \max (1, 5, 5, 5, 6, 6, 9, 10, 10) = 10$

FORMULA:

Min Filter

$$\hat{f}(x, y) = \min_{(s, t) \in S_{xy}} \{g(s, t)\}$$

- Min Filter mengganti nilai sebuah piksel di titik hotspot kernel dengan nilai terkecil dari gray-level dalam subimage.
- Min Filter berfungsi untuk mencari titik paling gelap dalam citra.
- Min Filter baik untuk menghilangkan **salt noise** yang memiliki nilai gray-level noise yang tinggi.

Citra *input* sebagai berikut :

5	6	5	7	8
9	10	1	3	2
5	6	10	3	2
7	4	4	5	1
2	3	5	3	3

Maka hasil *min filtering* adalah :

5	6	5	7	8
9	1	1	1	2
5	1	1	1	2
7	2	3	1	1
2	3	5	3	3

$F(2,2) = \min (1, 5, 5, 5, 6, 6, 9, 10, 10) = 1$

Max and Min Filters (Non-Linear Filters)



(a)

**Original Image with
Salt-and-pepper noise**



(b)

**Minimum filter removes
bright spots (maxima) and
widens dark image structures**



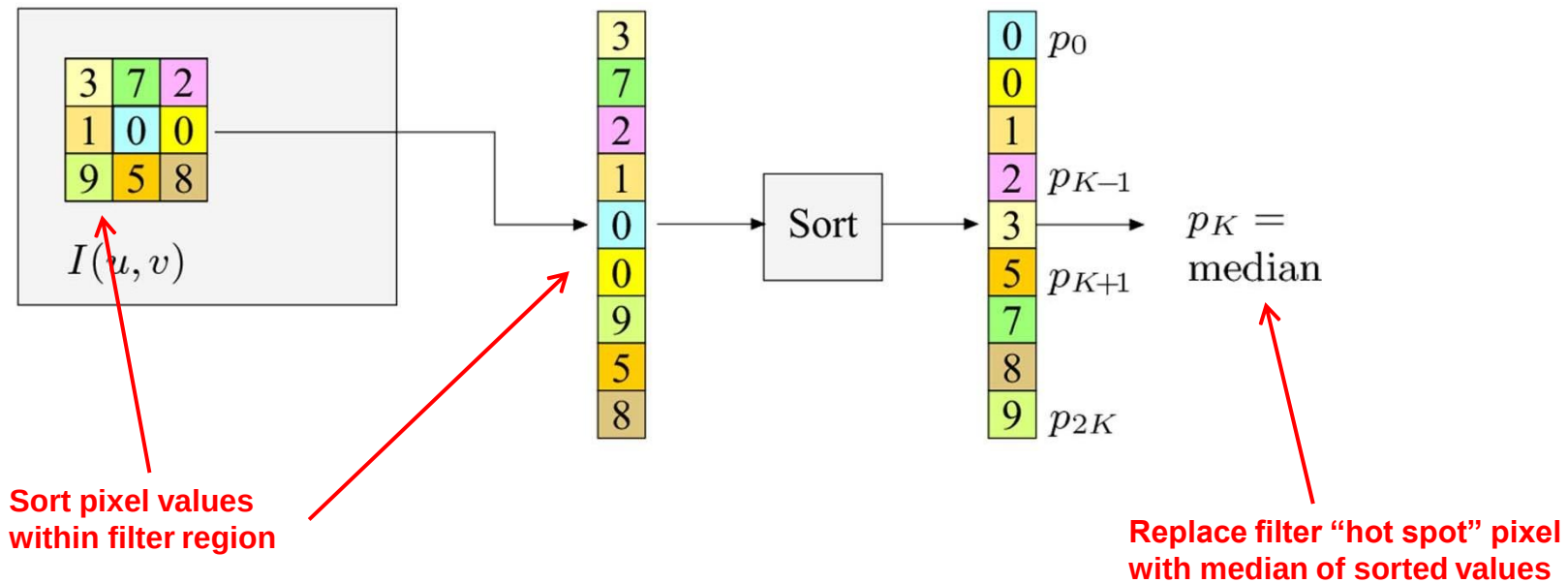
(c)

**Maximum filter (opposite effect):
Removes dark spots (minima) and
widens bright image structures**

Median Filter

- Much better at removing noise and keeping the structures

$$I'(u, v) \leftarrow \text{median} \{I(u+i, v+j) \mid (i, j) \in R\}$$

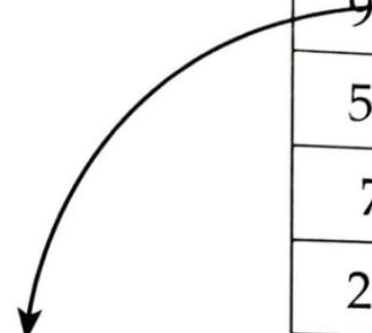


Example of Median Filter

Input :

5	6	5	7	8
9	10	1	3	2
5	6	10	3	2
7	4	4	5	1
2	3	5	3	3

Maka hasil *median filtering* adalah :

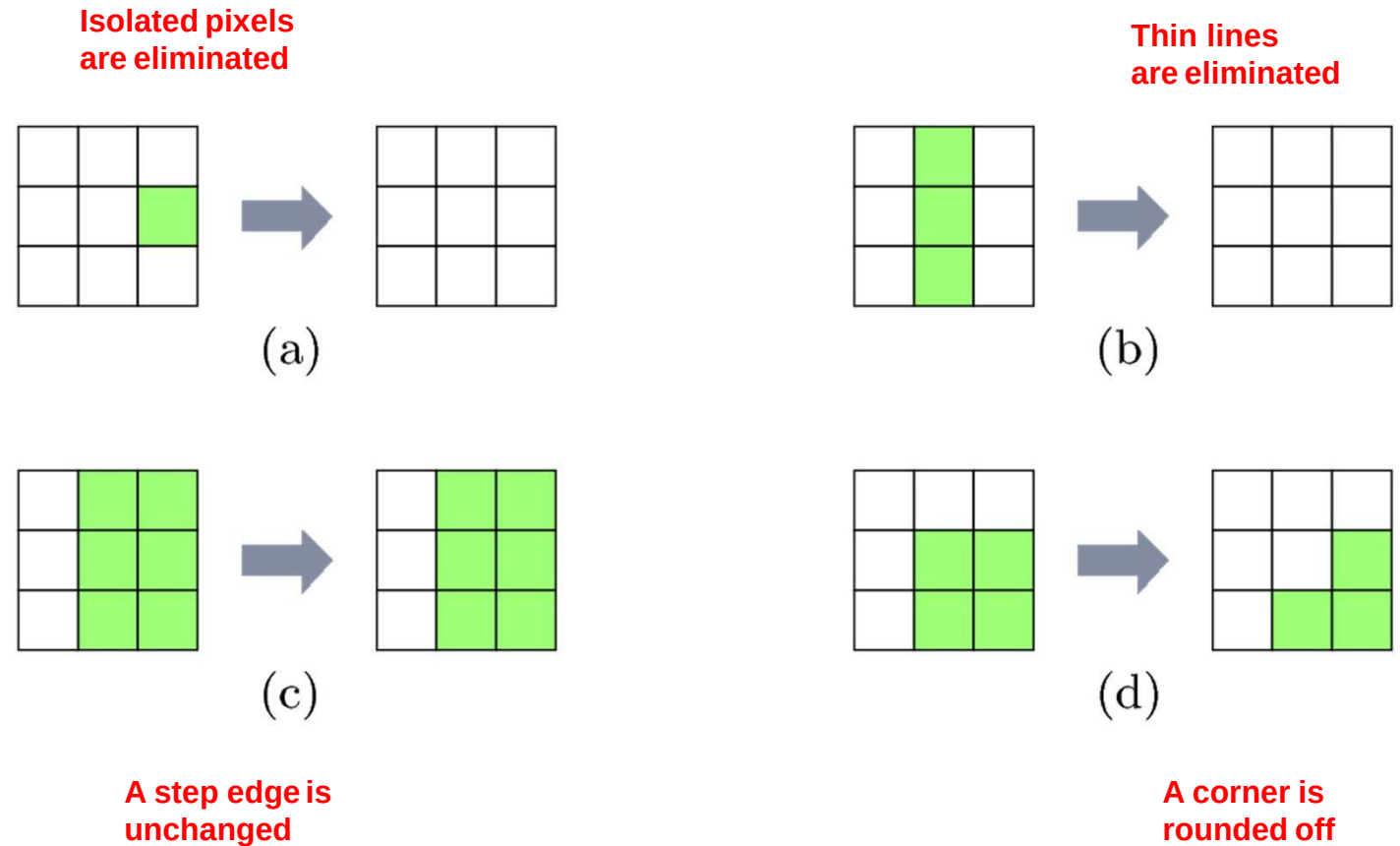


5	6	5	7	8
9	6	6	3	2
5	6	4	3	2
7	5	5	3	1
2	3	5	3	3

$$F(2,2) = \text{median}(1,5,5,5,6,6,9,10,10) = 6$$

Illustration: Effects of Median Filter

- Median filter memiliki kemampuan menghilangkan noise dengan lebih sedikit pengaburan dibandingkan filter lain dengan ukuran kernel yg sama.
- Baik untuk mereduksi **salt and pepper noise**.



Effects of Median Filter



(a)

**Original Image with
Salt-and-pepper noise**

(b)

**Linear filter removes some of
the noise, but not completely.
Smears noise**

(c)

**Median filter salt-and-pepper noise
and keeps image structures largely
intact. But also creates small spots
of flat intensity, that affect sharpness**

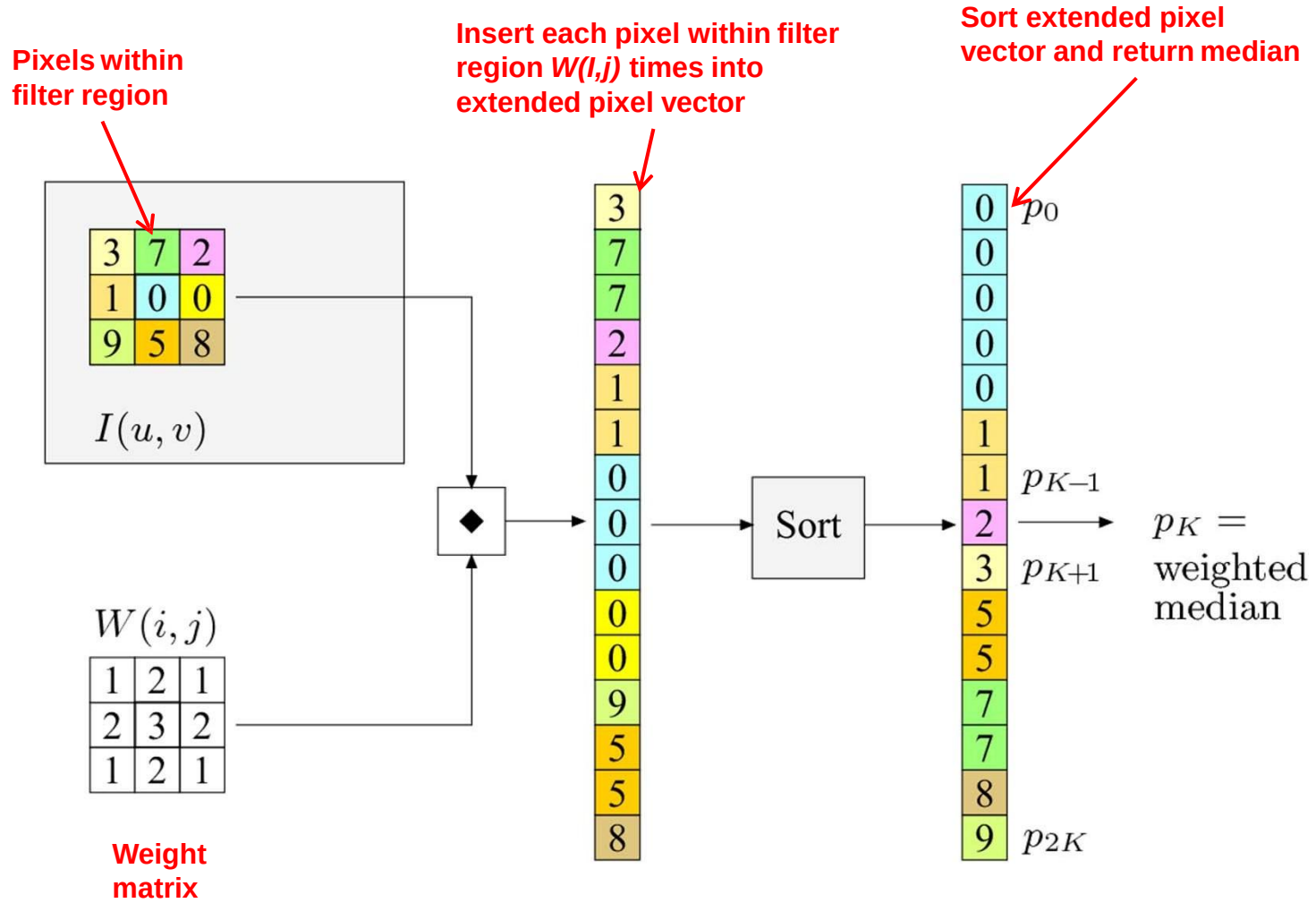
Weighted Median Filter (1)

- Color assigned by median filter determined by colors of “the majority” of pixels within the filter region
- Considered robust since single high or low value cannot influence result (unlike linear average)
- Median filter assigns weights (number of “votes”) to filter positions

$$W(i, j) = \begin{bmatrix} 1 & 2 & 1 \\ 2 & \mathbf{3} & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- To compute result, each pixel value within filter region is inserted $W(i, j)$ times to create **extended pixel vector**
- Extended pixel vector then sorted and median returned

Weighted Median Filter (2)



Note: assigning weight to center pixel larger than sum of all other pixel weights inhibits any filter effect (center pixel always carries majority)!!

Weighted Median Filter (3)

- More formally, **extended pixel vector** defined as

$$Q = (p_0, \dots, p_{L-1}) \quad \text{of length} \quad L = \sum_{(i,j) \in R} W(i,j)$$

- For example, following weight matrix yields extended pixel vector of length 15 (sum of weights)

$$W(i,j) = \begin{bmatrix} 1 & 2 & 1 \\ 2 & \mathbf{3} & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- Weighting can be applied to non-rectangular filters
- Example: *cross-shaped* median filter may have weights

$$W^+(i,j) = \begin{bmatrix} 0 & 1 & 0 \\ 1 & \mathbf{1} & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Filtering Function - Python

`scipy.ndimage.filters`

includes:

- `median_filter()`
- `uniform_filter()`
- `maximum_filter()`
- `percentile_filter()`

```
ndi.median_filter(im, size=10)
```



Other Non-Linear Filters

- Any filter operation that is not linear (summation), is considered linear
- Min, max and median are simple examples
- More examples later:
 - Morphological filters
 - Corner detection filters
- Also, filtering shall be discussed in frequency domain

Performance Evaluation in Image Processing

Performance Evaluation in Image Processing

- Penilaian hasil filter dapat dilakukan dengan dua cara :
 1. Penilaian visual: mudah, cepat, tetapi kurang obyektif.
 2. Penilaian parameter: obyektif karena menggunakan parameter penilaian. Contoh:
 - Equivalent number of looks (ENL)
 - Peak signal-to-noise ratio (PSNR)
 - Contrast-to-noise ratio (CNR)

Equivalent Number of Looks (ENL)

- The ability of smoothing speckle noise → ENL.
- ENL is an index to weigh the relative intensity that an image has **speckle noise**, and it reflects the capability for a filter to reduce speckle noise.
- **If ENL is higher, speckle noise is weaker in the image** and the interpretation is better.
- The definition of ENL is given by:

$$ENL = \frac{M^2}{VAR} = \frac{M^2}{SD^2}$$

where VAR denotes variance, M denotes mean value of intensity, SD denotes standard deviation.

Peak signal-to-noise ratio (PSNR)

- The PSNR is utilized to evaluate the performance of speckle noise, salt-pepper (or other noise) elimination.
- A high PSNR value indicates excellent speckle noise suppression ability.

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right)$$

Noisy
PSNR=16.51



Denoised: no cycle shifts
PSNR=26.38



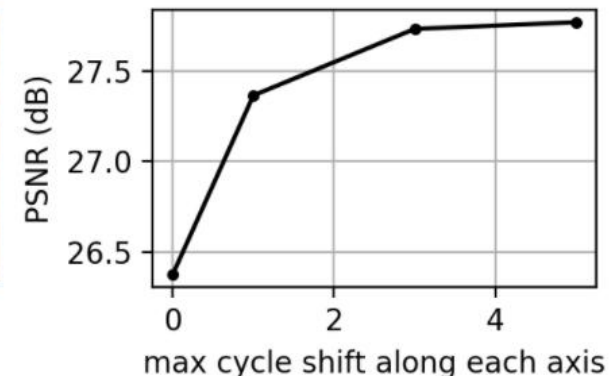
Denoised: 2x2 shifts
PSNR=27.36



Denoised: 4x4 shifts
PSNR=27.73



Denoised: 6x6 shifts
PSNR=27.77



Mean Square Error (MSE)

- The MSE represents the similarity between the original image and resulting image.
- A low MSE value denotes that the resulting image is close to the original image.

$$MSE = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \{f(M, N) - g(M, N)\}^2$$

where M and N are the number of rows and columns, respectively, f is the original image, and g is the result image obtained from the noisy image, applying the filtering method.

Contrast-to-noise ratio (CNR)

- CNR → measure **the contrast ratio** of the image and the quality of the image.

$$\text{CNR} = \frac{\text{Contrast}}{\text{Noise}} = \frac{|\mu_1 - \mu_2|}{\sqrt{\sigma_1^2 + \sigma_2^2}}$$

μ_1, μ_2, σ_1 and σ_2 in Equation are the expectation values and standard deviations of two different images.

Normalized Mean (NM)

- NM → The ability to save information.
- NM is a criterion to evaluate that the filter is whether or not unbiased estimation.
- If the NM approaches 1, it shows that the estimation is unbiased, which illuminates that the filtering method saves good original image information.
- The definition of NM is given by:

$$NM = \frac{M_{filtered}}{M_{original}}$$

- where $M_{filtered}$ and $M_{original}$ denote the mean value of pre-filter and post-filter for the same image, respectively.

Example

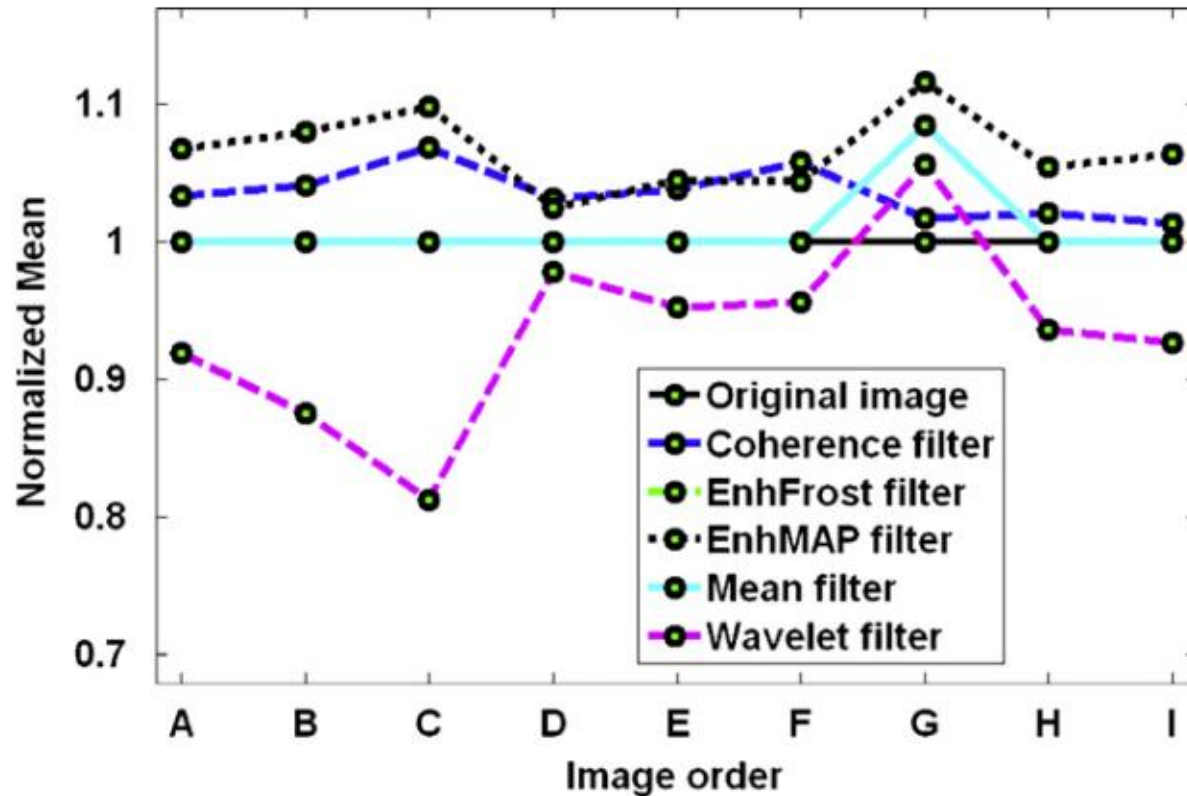


Fig. 8. NM comparisons of SAR images.

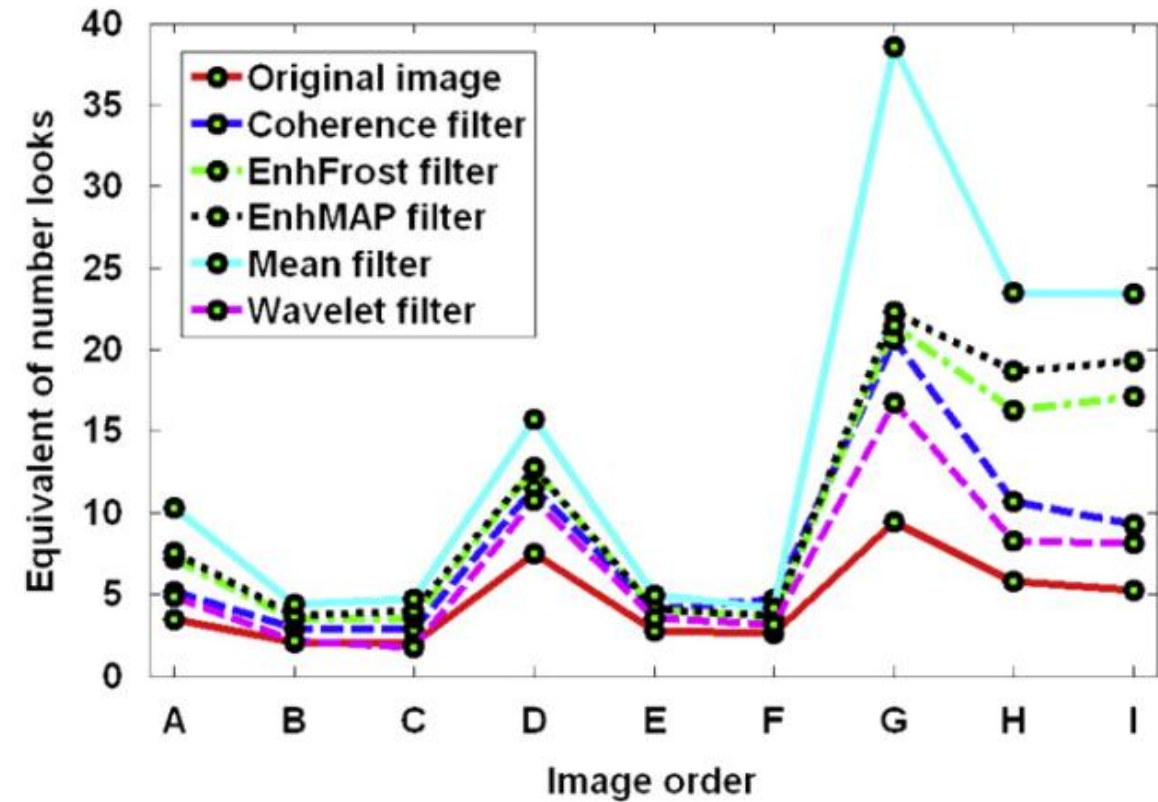


Fig. 9. ENL comparisons of the SAR images.

Source: Shi-qi Huang, Dai-zhi Liu, Gui-qing Gao, Xi-jian Guo, A novel method for speckle noise reduction and ship target detection in SAR images, *ELSEVIER Pattern Recognition*, 42 (2009) 1533 – 1542.